

Improved Key-recovery Attacks on ARADI

Orr Dunkelman^{1,2} and Shibam Ghosh^{3*}

¹Department of Computer Science, University of Haifa, Haifa, Israel.

²Faculty of Electrical Engineering and Computer Science, TU Berlin,
Berlin, Germany.

³Inria Paris, Paris, France.

*Corresponding author(s).

E-mail(s): shibam.ghosh@inria.fr;

Contributing authors: orrd@cs.haifa.ac.il;

Abstract

ARADI is a low-latency block cipher introduced by the U.S. National Security Agency (NSA), targeting secure and efficient memory encryption. However, unlike most academic cipher proposals, the design rationale behind **ARADI** has not been made public, leaving its security to be only assessed through independent analysis. In this work, we present improved key-recovery attacks on up to 12 out of 16 rounds of **ARADI** in the single-key setting — advancing the best known attacks by two rounds. Our techniques build upon the **ZeroSum** distinguisher framework and leverage the Fast Hadamard Transform (FHT). Following the weak diffusion of the linear layer of **ARADI**, our attacks enjoy a speed up in the partial decryption which requires only a subset of the round keys.

Keywords: ARADI, Lightweight Crypto, FHT-based attack, Integral Attack

1 Introduction

ARADI is a low-latency block cipher introduced by the U.S. National Security Agency (NSA) [GMW24]. It was proposed alongside a dedicated mode of operation, **LLAMA**, designed specifically to provide both confidentiality and integrity for computer memory — an increasingly critical requirement in modern computing environments. In contemporary processor architectures, it is common to protect the CPU within secure enclaves to safeguard sensitive computations. However, extending such protections to

the main memory (RAM) remains impractical due to physical and architectural constraints. This gap has made RAM a prominent target for physical attacks. For example, Cold Boot Attacks [HSH⁺08] have demonstrated how data remanence in RAM can be exploited to extract cryptographic keys and other sensitive information.

To defend against such threats, cryptographic memory protection schemes have been proposed. These involve encrypting RAM contents and performing integrity checks [AMS⁺22]. Designing such systems, however, presents a range of challenges, chief among them is achieving strong security without compromising performance. In particular, memory encryption schemes must avoid introducing latency in memory accesses. This requirement has driven the demand for *low-latency* block ciphers.

As a result, recent years have seen the development of several block ciphers explicitly designed for this setting. Notable examples include PRINCE [BCG⁺12] and its successor PRINCEv2 [BEK⁺20], both designed to offer minimal latency. Tweakable block ciphers, which offer additional flexibility for applications like memory encryption, have also gained prominence. Examples include MANTIS [BJK⁺16], QARMA [Ava17], and the more recent QARMAv2 [ABD⁺23]. Beyond RAM encryption, low-latency tweakable block ciphers are also being used in other memory-related security applications. For instance, CHILOW [BDG⁺25] was designed for embedded code encryption, while SCARF [CGL⁺23] targets cache address randomization to defend against side-channel attacks targeting cache memory. All of these ciphers share a common design philosophy: prioritizing ultra-low latency as a primary goal. ARADI follows this same principle. It is designed with a 128-bit block size and a 256-bit key size, tailored to meet the demands of secure, low-latency memory encryption.

In this paper, we propose improved key-recovery attacks on ARADI using ZeroSum distinguishers. The two important methods for key-recovery using ZeroSum distinguishers (also known as integral attacks) are the *partial sum technique* and the *Fast Hadamard Transform* (FHT)-based algorithm. The partial sum attack was proposed by Ferguson et al. [FKL⁺00] to improve the SQUARE attack on 6-round of AES [DR00]. Whereas, the foundational idea of employing FHT in the context of key-recovery attacks was first introduced by Collard et al. in [CSQ07], originally in the setting of linear cryptanalysis, marking an important milestone in the practical use of FHT-based cryptanalysis. This concept was later adapted by Todo and Aoki [TA14], who applied it to ZeroSum distinguishers to successfully attack 6-round of AES [DR00] with a comparable complexity as the partial sum attack in [FKL⁺00]. More recently, Dunkelman et al. advanced this line of research by combining the FHT with the partial-sum method to improve attacks on 6-round AES in [DGK⁺24]. This hybrid approach further illustrates the versatility and effectiveness of FHT framework in reducing the computational complexity of key-recovery attacks. Our key-recovery techniques are built upon this modern combined approach of [DGK⁺24].

1.1 Related Cryptanalysis on ARADI

Since its release, ARADI has attracted some attention from the cryptographic research community, leading to several preliminary analyses. In [BFG⁺24], Bellini et al. conducted an extensive evaluation of ARADI using automated tools to identify various distinguishers. Their work included statistical tests (such as avalanche and diffusion

analysis), differential and linear distinguishers up to 9 rounds, impossible differentials up to 8 rounds, and ZeroSum distinguishers up to 7 rounds. This line of research was further advanced in [BRRT25], where Bellini et al. extended the ZeroSum distinguisher to 8 rounds and introduced a key-recovery attack on 10 rounds of ARADI, leveraging the 8-round distinguisher. In the same work, they also proposed a structural distinguisher covering up to 8 rounds, which achieves slightly lower complexity than the corresponding ZeroSum-based approach. Additionally, [BRRT25] includes an experimental observation of a byte-wise equality pattern in the cube sum for 5 rounds of ARADI. However, this finding lacked a theoretical explanation at the time. Later, in [KKL⁺24], Kim et al. provided a theoretical foundation for this phenomenon using the division property model introduced in [LDF20].

In a related work [HJ25], Hu et al. recently proposed a key-recovery attack on ARADI based on impossible boomerang distinguishers — however, this attack is in the related-key setting. Meanwhile, Kwon et al. explored side-channel vulnerabilities in ARADI [KH24]. Beyond the cipher itself, LLAMA—the mode of operation paired with ARADI—was also scrutinized. In [ADG24], Avanzi et al. highlighted a potential issue related to the use of variable-length initialization vectors. This concern was acknowledged and subsequently addressed by the ARADI designers [GMW24].

1.2 Contributions

In contrast to most academic proposals for new symmetric cryptographic primitives, the design rationale behind ARADI was not made publicly available by its authors. This stands in contrast to the standard practice in the cryptographic community, where detailed justifications of design choices are provided to aid in evaluation and foster trust. As a result, assessing the security of ARADI demands a comprehensive, independent cryptanalytic investigation across a wide range of attack models. The absence of a publicly documented rationale has been a key motivation for our work, prompting us to examine the cipher through the lens of key-recovery attacks.

Our work presents a series of advanced key-recovery attacks on ARADI, building upon the recent ZeroSum distinguishers of Bellini et al. [BRRT25]. The core methodology behind key-recovery attacks using ZeroSum distinguishers relies on three main steps: (i) guessing selected portions of the round key, (ii) partially decrypting the corresponding ciphertexts, and (iii) checking whether the XOR sum of the partially decrypted values evaluates to zero on some selected bits. We adapt the powerful hybrid partial sum and FHT-based technique from Dunkelman et al. [DGK⁺24] to efficiently perform these steps. The basic idea is to use the general structure of partial sums during partial decryption, but replace combinations of several such steps using the *Hadamard Transform* to use computational benefits of FHT.

However, the main hurdle of such attack is to choose which steps are combined together and when to use FHT to get optimal applicability. The strategy for combining steps depends heavily on parameters like subkey guesses, pre-computation potential, and partial knowledge of some subkey. Finding the *right* combination of steps becomes increasingly difficult with each additional round of partial decryption. This problem is magnified in our attack, as we leverage ARADI’s slow diffusion to perform partial decryption over up to four rounds. In addition, this offers new insights into the ARADI

Table 1 Comparison of our results with prior work on key-recovery attacks on ARADI.

Rounds	Distinguisher	Time	Data	Memory (bits)	Reference
10	ZeroSum	2^{192}	2^{124}	2^{131}	[BRRT25] [†]
10	ZeroSum	2^{125}	2^{125}	2^{76}	Section 3.1
10		2^{118}	2^{118}	2^{48}	Section 3.2
11		2^{126}	2^{126}	2^{48}	Section 3.3
11		$2^{150.4}/2^{147.7^*}$	2^{119}	$2^{107.13}$	Section 3.4
12		$2^{154.48}/2^{151.8^*}$	$2^{126.8}$	$2^{123.5}$	Section 3.5
13	Impossible Differential	2^{255}	$2^{121.47}$	2^{112}	[DE25] [‡]
14	Differential	$2^{247.57}$	2^{128}	$2^{178.58}$	[GMNP25] [‡]
14		$2^{248.57}$	2^{126}	$2^{178.58}$	

*: Improved time complexity using the packing technique.

†: The authors of [BRRT25] report a time complexity of 2^{177} for their key-recovery attack on 10 rounds of ARADI, which includes a brute-force step of 2^{64} . However, we note that the brute-force step should, in fact, require 2^{192} encryptions. This is because the ZeroSum distinguisher used in [BRRT25] extracts only 2 bits of key information per check, and after 32 such checks, 64 bits information of the master key are recovered. Recovering the remaining 192 bits would thus necessitate an exhaustive search over 2^{192} remaining candidates.

‡: These attacks were developed independently, and were uploaded after our work became public on ePrint [DG25].

’s internal structure and demonstrates how to attack more rounds using ZeroSum distinguishers rather than previously known. We note that two later and independent works of [DE25] and [GMNP25] were able to attack 13 and 14 rounds of ARADI.

Furthermore, we extensively exploit ARADI’s linear key schedule to enhance our attacks, which was not considered in previous works. Specifically, we present attacks on up to 12 out of 16 rounds of ARADI in the single-key setting — advancing the state of the art by two rounds beyond the best known key-recovery attacks of [BRRT25] using ZeroSum distinguisher.

Finally, we demonstrate that the efficiency of some of our attack can be further improved by incorporating the packing technique from [DGK⁺24], which parallelizes multiple FHT computations. A detailed comparison of our results with prior work is summarized in Table 1.

1.3 Discussion on the Key-Recovery Strategies

A key observation of our work is that partial decryption (which requires key guessing) can be carried out across up to four rounds without requiring the full round keys. This is made possible due to a crucial structural weakness in ARADI ’s design: After 3.5 rounds (3 full rounds and one SBox layer), ARADI does not achieve *full diffusion* in both forward and backward directions. This limitation stems from the inherently low diffusion of the linear layer of ARADI in both forward and backward directions. Each

linear transformation within ARADI is implemented as a 32-bit binary matrix operation, with matrices carefully chosen to support low-latency execution. Specifically, all these matrices have a row density of three, meaning that each output bit depends on only three input bits, and more importantly, only two XOR operations are required per output bit — an optimization critical for fast hardware implementations. Moreover, these matrices are involutive, meaning that each matrix is its own inverse. Consequently, the inverse of the linear layer (required for partial decryption) also has a low diffusion and can be implemented with two XOR operations per output bit. This symmetrical and sparse structure leads to a slow diffusion across rounds, which in turn enables our attacks to extend the distinguisher through multiple rounds of partial decryption. This observation brings to light a fundamental design challenge: how can a linear layer be constructed to offer both low latency and strong diffusion, thereby achieving a more balanced trade-off between performance and security?

Our findings underline the importance of carefully analyzing the diffusion behavior of linear layers in low-latency ciphers. While latency-optimized structures are critical for performance, they must also be scrutinized for their long-term resistance to structural attacks such as the ones presented here.

2 Background

2.1 Notations

For any n -bit vector $x \in \mathbb{F}_2^n$, we denote its i -th coordinate by x_i , so $x = (x_0, \dots, x_{n-1})$. Given two vectors $u, x \in \mathbb{F}_2^n$, we define the monomial x^u as $x^u = \prod_{i=1}^n x_i^{u_i}$. An n -variable Boolean function f is a mapping from $\mathbb{F}_2^n$ to $\mathbb{F}_2$. The *Algebraic Normal Form* (ANF) of a Boolean function f is given by: $f(x) = \bigoplus_{u \in \mathbb{F}_2^n} a_u^f x^u$. The degree of a Boolean function $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$, denoted $\deg(f)$, is the degree of the highest-degree monomial in its ANF. That is,

$$\deg(f) = \max_{u \in \mathbb{F}_2^n, a_u^f \neq 0} wt(u),$$

where $wt(u)$ is the Hamming weight of u . A vectorial Boolean function is defined as:

$$\begin{cases} f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m \\ x = (x_0, x_1, \dots, x_{n-1}) \mapsto (f_0(x), f_1(x), \dots, f_{m-1}(x)), \end{cases}$$

where each f_i is a Boolean function from $\mathbb{F}_2^n$ to $\mathbb{F}_2$, referred to as the coordinate functions of f . The degree of a vectorial Boolean function F is defined as the maximum degree among its coordinate functions:

$$\deg(f) = \max_{i \in \{0, \dots, m-1\}} \deg(f_i).$$

2.2 Specification of ARADI

We briefly revisit the design of ARADI [GMW24] in this section. The ARADI cipher follows a *Substitution-Permutation Network* (SPN) structure. It operates on 128-bit data blocks, and supports a 256-bit master key.

State Representation of ARADI. The internal state of ARADI is a 128-bit vector over $\mathbb{F}_2$, i.e., an element of $\mathbb{F}_2^{128}$. This state can be conceptually arranged as a 4×32 matrix of bits for ease of representation and manipulation. For instance, the 128-bit state x can be visualized as:

$x_{0,0}$	$x_{0,1}$	$x_{0,2}$	$x_{0,3}$	$x_{0,4}$	$x_{0,5}$	$x_{0,6}$	$x_{0,7}$	$x_{0,8}$	$x_{0,9}$	$x_{0,10}$	$x_{0,11}$	$x_{0,12}$	$x_{0,13}$	$x_{0,14}$	$x_{0,15}$	$x_{0,16}$	$x_{0,17}$	$x_{0,18}$	$x_{0,19}$	$x_{0,20}$	$x_{0,21}$	$x_{0,22}$	$x_{0,23}$	$x_{0,24}$	$x_{0,25}$	$x_{0,26}$	$x_{0,27}$	$x_{0,28}$	$x_{0,29}$	$x_{0,30}$	$x_{0,31}$
$x_{1,0}$	$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$	$x_{1,5}$	$x_{1,6}$	$x_{1,7}$	$x_{1,8}$	$x_{1,9}$	$x_{1,10}$	$x_{1,11}$	$x_{1,12}$	$x_{1,13}$	$x_{1,14}$	$x_{1,15}$	$x_{1,16}$	$x_{1,17}$	$x_{1,18}$	$x_{1,19}$	$x_{1,20}$	$x_{1,21}$	$x_{1,22}$	$x_{1,23}$	$x_{1,24}$	$x_{1,25}$	$x_{1,26}$	$x_{1,27}$	$x_{1,28}$	$x_{1,29}$	$x_{1,30}$	$x_{1,31}$
$x_{2,0}$	$x_{2,1}$	$x_{2,2}$	$x_{2,3}$	$x_{2,4}$	$x_{2,5}$	$x_{2,6}$	$x_{2,7}$	$x_{2,8}$	$x_{2,9}$	$x_{2,10}$	$x_{2,11}$	$x_{2,12}$	$x_{2,13}$	$x_{2,14}$	$x_{2,15}$	$x_{2,16}$	$x_{2,17}$	$x_{2,18}$	$x_{2,19}$	$x_{2,20}$	$x_{2,21}$	$x_{2,22}$	$x_{2,23}$	$x_{2,24}$	$x_{2,25}$	$x_{2,26}$	$x_{2,27}$	$x_{2,28}$	$x_{2,29}$	$x_{2,30}$	$x_{2,31}$
$x_{3,0}$	$x_{3,1}$	$x_{3,2}$	$x_{3,3}$	$x_{3,4}$	$x_{3,5}$	$x_{3,6}$	$x_{3,7}$	$x_{3,8}$	$x_{3,9}$	$x_{3,10}$	$x_{3,11}$	$x_{3,12}$	$x_{3,13}$	$x_{3,14}$	$x_{3,15}$	$x_{3,16}$	$x_{3,17}$	$x_{3,18}$	$x_{3,19}$	$x_{3,20}$	$x_{3,21}$	$x_{3,22}$	$x_{3,23}$	$x_{3,24}$	$x_{3,25}$	$x_{3,26}$	$x_{3,27}$	$x_{3,28}$	$x_{3,29}$	$x_{3,30}$	$x_{3,31}$

Each element $x_{i,j}$ in this matrix corresponds to the $(32 \cdot i + j)$ -th bit of the state x . We denote the j -th column of the state x as $x_{\uparrow j}$, which consists of the bits $(x_{0,j}, \dots, x_{3,j})$, and the i -th row as $x_{i \rightarrow}$, which consists of the bits $(x_{i,0}, \dots, x_{i,31})$. Furthermore, we denote the concatenation of the columns $x_{\uparrow i_0}, \dots, x_{\uparrow i_k}$ as $x_{\uparrow i_0, \dots, i_k}$.

2.2.1 Round Function of ARADI

The round function of ARADI comprises of three main steps: the substitution (SBox) layer, the linear layer, and the round key addition. We adopt the same notation as used in the original specification paper [GMW24]. Specifically:

- The SBox layer is denoted by π ,
- The i -th linear layer is represented by Λ_i , and
- The XOR with the 128-bit round key, denoted by τ_{k^i} , where k^i is the round key.

With these definitions, the i -th round function of ARADI, can be expressed as: $\mathcal{R}_i = \tau_{k^i} \circ \Lambda_i \circ \pi$, where k^i is the i -th round key. Additionally, there is an initial key addition — commonly referred to as “pre-whitening” — applied before the first round. This step is denoted by τ_{k^0} . The cipher operates over 16 rounds in total. Using the above notations, the full encryption process of ARADI can be described as:

$$(\tau_{k^{16}} \circ \Lambda_{16} \circ \pi) \circ \dots \circ (\tau_{k^1} \circ \Lambda_1 \circ \pi) \circ \tau_{k^0}.$$

Remark 1. We note that, in our description, the round key is added at the end of each round, rather than at the beginning as done in [GMW24]. This adjustment simplifies the presentation of our attacks. Functionally, this change is equivalent, since we include a pre-whitening key addition before the first round instead of a post-whitening key addition after the final round.

Table 2 The 4-bit SBox S and its inverse S^{-1} of ARADI.

S	0	1	2	3	4	D	F	6	8	B	5	E	C	7	A	9
S^{-1}	0	1	2	3	4	A	7	D	8	F	E	9	C	5	B	6

Table 3 The rotation offsets of the linear layer of ARADI.

$j = (i - 1) \bmod 4$	a_j	b_j	c_j
0	11	8	14
1	10	9	11
2	9	4	14
3	8	9	7

The SBox Layer of ARADI (π). The SBox layer applies a fixed 4-bit substitution box S to each of the 32 columns of the internal state in parallel. That is, the same SBox function is applied independently to all columns. The lookup tables for the 4-bit SBox S and its inverse S^{-1} are provided in Table 2.

The linear layer of ARADI (Λ). The linear layer of ARADI updates each row of the internal state by applying a round-dependent linear transformation. Specifically, at the i -th round, the linear layer Λ_i transforms the state x as follows:

$$\Lambda_i(x) = (L_j(x_{0\rightarrow}), L_j(x_{1\rightarrow}), L_j(x_{2\rightarrow}), L_j(x_{3\rightarrow})), \text{ for } i = 1, 2, \dots, 16,$$

where ¹ $j = (i - 1) \bmod 4$. Each L_0, L_1, L_2, L_3 is an involutive linear transformation over $\mathbb{F}_2^{32}$. To define L_j more precisely, let each 32-bit row be viewed as the concatenation of two 16-bit parts: u (upper half) and l (lower half). Then, the transformation L_j is given by:

$$L_j(u||l) = (u \oplus (u \lll a_j) \oplus (l \lll c_j) || (l \oplus (u \lll a_j) \oplus (l \lll b_j)),$$

where $\lll$ denotes a left rotation, and the rotation offsets a_j , b_j , and c_j are defined in Table 3.

2.2.2 The key schedule of ARADI

The master key K of ARADI is 256 bits and is represented as an array of eight 32-bit words. We denote the key schedule's internal state at round i by

$$K^i = (K_0^i, K_1^i, \dots, K_7^i).$$

¹Note that the indexing for the linear layer begins at 0, while round indices begin at 1.

Initially, the key state is loaded directly from the master key, i.e.,

$$K^0 = K = (K_0^0, K_1^0, \dots, K_7^0).$$

The 128-bit pre-whitening key k^0 and the 128-bit round keys k^i , for $i = 1, \dots, 16$, are derived from the key schedule state by concatenating four 32-bit words as follows:

$$k^i = \begin{cases} K_0^i || K_1^i || K_2^i || K_3^i, & \text{if } i \bmod 2 = 0 \\ K_4^i || K_5^i || K_6^i || K_7^i, & \text{if } i \bmod 2 = 1. \end{cases}$$

Then, the key state at the i -th step is updated through the following sequence of operations:

Linear Transformations: At first, two invertible linear maps, M_0 and M_1 , each operating on pairs of 32-bit words, are applied to the key state. These maps are defined as:

$$\begin{aligned} M_0(x, y) &= ((x \lll 1) \oplus y, (y \lll 3) \oplus (x \lll 1) \oplus y) \\ M_1(x, y) &= ((x \lll 9) \oplus y, (y \lll 28) \oplus (x \lll 9) \oplus y) \end{aligned}$$

The map M_0 is applied to $K_0^i || K_1^i$ and $K_4^i || K_5^i$ while M_1 is applied to $K_2^i || K_3^i$ and $K_6^i || K_7^i$.

Word Permutation: Next, a permutation P is applied to the eight words K^i , defined as

$$P = \begin{cases} (1, 2)(5, 6), & \text{if } i \bmod 2 = 0 \\ (0, 3)(4, 7), & \text{if } i \bmod 2 = 1 \end{cases}$$

Round Constant Addition: Finally, the step index i is XOR-ed as a round constant to the word K_7^i .

We note that ARADI's key schedule is an affine transformation. Hence, all the round keys are an affine combination of the master key bits. We will use this property of the key schedule extensively to enhance our attacks.

2.2.3 Equivalent Representation of ARADI

To facilitate the description of our attacks, we adopt an equivalent representation of ARADI, where the order of the linear layer Λ_i and the round key addition τ_{k_i} is swapped. Each round of ARADI can be expressed as

$$\mathcal{R}_i = \tau_{k^i} \circ \Lambda_i \circ \pi = \Lambda_i \circ \tau_{u^i} \circ \pi, \text{ where } u^i = \Lambda_r(k^i).$$

Using this equivalence, the full cipher can be represented as

$$(\Lambda_{16} \circ \tau_{u^{16}} \circ \pi) \circ \dots \circ (\Lambda_1 \circ \tau_{u^1} \circ \pi) \circ \tau_{k^0}.$$

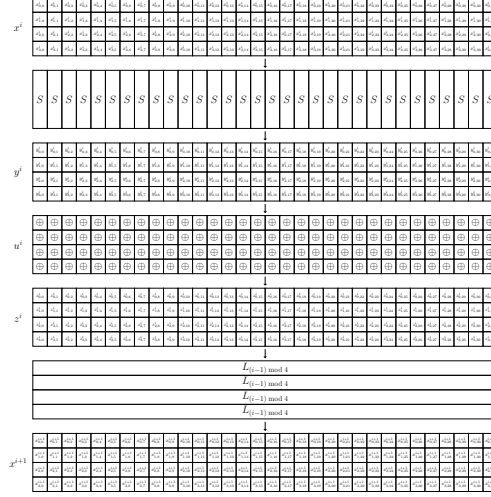


Fig. 1 The i -th round function of ARADI (the order of Λ_i and τ_{k^i} is swapped)

232 Since we have interchanged the order of Λ_i and τ_{k^i} , the cipher's final operation becomes
 233 a linear layer, which we can safely omit in our attacks.

234 Next, we introduce notation for the internal states in this equivalent representation.
 235 Let, x^i and x^{i+1} denote the input and output states of the i -th round, respectively.
 236 The intermediate states after the SBox layer and the round key addition within the
 237 i -th round is denoted by y^i and z^i , respectively. These notations are illustrated in
 238 Figure 1.

239 2.3 ZeroSum Distinguishers of ARADI

240 We begin by briefly recalling the concept of ZeroSum distinguishers [AM09, BC10,
 241 BCC11]. Consider a Boolean function $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$. The Boolean derivative of f with
 242 respect to a vector a is defined as:

$$\Delta_a f(x) = f(x \oplus a) \oplus f(x).$$

243 This notion naturally generalizes to higher-order derivatives, obtained by applying
 244 the derivative multiple times with respect to a sequence of vectors $a_1, a_2, \dots, a_t$.
 245 Recursively, this is expressed as:

$$\Delta_{a_1, a_2, \dots, a_t} f(x) = \Delta_{a_1, a_2, \dots, a_{t-1}} (f(x \oplus a_t) \oplus f(x)).$$

246 Lai [Lai94] showed that if the vectors $a_1, a_2, \dots, a_t$ form a basis for a vector space $\mathcal{V}$,
 247 then the higher-order derivative simplifies to

$$\Delta_{a_1, a_2, \dots, a_t} f(x) = \bigoplus_{v \in \mathcal{V}} f(x \oplus v).$$

248 Consequently, for a Boolean function $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$ and a vector space $\mathcal{V}$, the derivative
 249 of f with respect to $\mathcal{V}$ can be defined as

$$\Delta_{\mathcal{V}}f(\mathbf{x}) = \bigoplus_{v \in \mathcal{V}} f(x \oplus v).$$

250 Furthermore, applying Boolean derivative with respect to a unit vector e_i is equivalent
 251 to taking a partial derivative with respect to the variable x_i . This operation removes
 252 all monomials x^u that do not contain x_i from the ANF of f , and reduces the degree
 253 of those that do, leading to the following important fact:

$$\Delta_{\mathcal{V}}f(x) = \begin{cases} 0, & \text{if } \deg(f) < \dim \mathcal{V} \\ \text{constant}, & \text{if } \deg(f) = \dim \mathcal{V}. \end{cases}$$

254 Hence, if the algebraic degree of a Boolean function f is less than d , there exists an
 255 affine vector space $\mathcal{V}$ of dimension d such that

$$\bigoplus_{\mathbf{x} \in \mathcal{V}} f(\mathbf{x}) = 0.$$

256 This property always holds for such functions, whereas for a random Boolean function,
 257 it holds only with probability $1/2$. This can be exploited to distinguish f from a random
 258 function by selecting an appropriate subspace of dimension d . Such a distinguisher
 259 is known as a **ZeroSum** distinguisher [AM09, BC10, BCC11]. Since the size of the
 260 subspace is $2^{\dim \mathcal{V}}$, the data complexity of the distinguisher is $2^{\dim \mathcal{V}}$.

261 This idea extends naturally to vectorial Boolean functions $F : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, where
 262 the derivatives are considered coordinate-wise on each output bit. In particular, if the
 263 degree of any coordinate function of F is less than d , a **ZeroSum** distinguisher can
 264 be constructed. In this context, the corresponding output bit is called a **balanced**
 265 **bit**. Moreover, the vector space $\mathcal{V}$ is typically generated by inputs that vary over all
 266 possible combinations of a subset of bits of size $\dim \mathcal{V}$, while the remaining input bits
 267 are held fixed. We denote such a vector space as a **cube** and the bits that vary freely
 268 over all possible values are referred to as **cube variables**.

269 In our attacks on **ARADI**, we revisit several **ZeroSum** distinguishers originally pro-
 270 posed in [BRRT25]. In that work, the authors employed the three-subset bit-based
 271 division property (3SBDP) [HLM⁺20] to establish degree bounds for up to 8 rounds of
 272 **ARADI**. According to the findings in [BRRT25], after 7 rounds of **ARADI**, the algebraic
 273 degree is bounded by 116 for each bit in the 0-th and 2-nd rows ($x_{0\rightarrow}^8$ and $x_{2\rightarrow}^8$), while
 274 the degrees of the bits in the 1-st and 3-rd rows ($x_{1\rightarrow}^8$ and $x_{3\rightarrow}^8$) are bounded by 115.
 275 Similarly, for 8 rounds, they reported a degree bound of 124 for all bits in the 0-th
 276 and 2-nd rows ($x_{0\rightarrow}^9$ and $x_{2\rightarrow}^9$), and 123 for all bits in the 1-st and 3-rd rows ($x_{1\rightarrow}^9$ and
 277 $x_{3\rightarrow}^9$). Details about the selected cube variables as well as the degrees are summarized
 278 in Table 4. Building on this 8-round distinguisher, the authors in [BRRT25], proposed
 279 a key-recovery attack on 10 rounds of **ARADI**.

280 However, while the cube used in [BRRT25] is of dimension 124, this does not fully
 281 align with the theoretical requirement. Specifically, to guarantee the **ZeroSum** property

Round	Cube Variables	Cube Dimension	$\deg(x_{0\rightarrow}),$ $\deg(x_{2\rightarrow})$	$\deg(x_{1\rightarrow}),$ $\deg(x_{3\rightarrow})$	Ref.
7		116	116	115	[BRRT25]
8		124	124	123	[BRRT25]

Table 4 Cube variables (■) and cube dimensions of the distinguishers for 7 and 8 rounds of ARADI.

when the algebraic degree is upper-bounded by 124, the cube should have a dimension of at least 125. As a result, the 124-dimensional cube described in [BRRT25] ensures balanced bits only in rows $x_{1\rightarrow}^9$ to $x_{3\rightarrow}^9$ — not across all four rows. This implies that the distinguisher used in [BRRT25] for the 10-round attack offers a distinguishing advantage of 2 balanced bits, instead of 4 balanced bits, as claimed. Accordingly, we have revised the reported time, data, and memory complexities of the attack in [BRRT25], as reflected in Table 1. That said, the issue can be addressed by using two independent cubes instead of one, which restores the full distinguishing advantage — at the cost of doubling the data complexity.²

2.4 FHT-Based Key-Recovery Attacks

Here we present a high-level overview of the general attack strategy for Fast Hadamard Transform (FHT) based key-recovery attacks. Once a ZeroSum distinguisher is found, it can be exploited for key-recovery. The fundamental approach involves guessing portions of the relevant round keys, partially decrypting the corresponding ciphertexts, and verifying whether the XOR sum of the partially decrypted values equals zero on some selected bits. We describe the general framework of computing such sums using FHTs in the following.

Consider a function $F : \mathbb{F}_2^n \times \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ which is defined by another function f as $F(u, c) = f(u \oplus c)$. This structure is typical in many block ciphers, where round keys are XOR-ed with the internal state during encryption. Also, consider a binary array A , where our goal is to compute the following sum for each guess of $u \in \{0, 1\}^n$:

$$\text{sum}[u] = \bigoplus_{\{c: A[c]=1\}} f(u \oplus c). \quad (1)$$

Next, define the indicator function $g : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$ for the array A as follows:

$$g(c) = \begin{cases} 1, & \text{if } A[c] = 1 \\ 0, & \text{if } A[c] = 0. \end{cases}$$

²This resolution has been confirmed through personal communication with the authors of [BRRT25].

304 Then, for each guess of u , the sum from Equation (1) can be rewritten as:

$$\begin{aligned}
\text{sum}[u] &= \bigoplus_{\{c: A[c]=1\}} f(u \oplus c) \\
&= \bigoplus_{c \in \{0,1\}^n} g(c) f(u \oplus c) \\
&= \left(\sum_{c \in \{0,1\}^n} g(c) f(u \oplus c) \right) \bmod 2 \\
&= ((g * f)(u)) \bmod 2.
\end{aligned}$$

305 This formulation reveals the core idea behind the FHT-based approach: the values
306 of $\text{sum}[u]$ for all u correspond to the convolution of two functions of size 2^n . Instead
307 of computing each value independently, we can evaluate all 2^n convolutions simulta-
308 neously using a matrix-vector multiplication — an operation that can be efficiently
309 performed with the Fast Hadamard Transform (FHT). Let M be the matrix where the
310 element at the u -th row and a -th column is defined by: $M[u, a] = f(u \oplus a)$. To compute
311 the value associated with a key candidate u , we need to perform the matrix-vector
312 multiplication $M \times A$. Our main motivation is to replace this costly matrix-vector
313 multiplication with a more efficient computation using a small number of FHTs. A key
314 insight — originally used by Collard et al. in [CSQ07] in the context of linear crypt-
315 analysis — is that for this class of functions, where the round keys are XOR-ed with
316 the internal state, the matrix M is diagonalizable by a special discrete Fourier matrix
317 known as the *Hadamard matrix*. This observation is formalized in the following lemma.

318 **Lemma 1** ([CSQ07]). *Let $f : \mathbb{F}_2^m \times \mathbb{F}_2^m \rightarrow \mathbb{F}_2$ be a Boolean function and the matrix*
319 *M is constructed from f whose entries are of the form*

$$M[u, a] = f(u \oplus a).$$

320 *This matrix diagonalizes as $M = 1/2^m H_m \mathcal{D} H_m$ where H_m is the Hadamard matrix*
321 *of size 2^m whose entries are $h_{u,a} = (-1)^{u \cdot a}$, and $\mathcal{D}$ is a diagonal matrix whose*
322 *diagonal elements are the eigenvalues of M . Moreover, the eigenvalue vector of M*
323 *can be computed by multiplying the first column of M by H_m .*

324 However, in our case the matrix function from which M is constructed is not a
325 Boolean function, but a vectorial Boolean function. Thus, we can not apply the lemma,
326 as FHT must be calculated over integers. Todo et al. [TA14] proposed to consider the
327 coordinate functions of $f = (f_0, f_1, \dots, f_{n-1})$, one by one. Thus, we define n binary
328 matrices as $M_j[u, a] = f_j(u \oplus a)$, for each coordinate $j \in \{0, 1, \dots, n-1\}$ and we
329 compute n matrix-vector multiplications, $(M_0 \times A), (M_1 \times A), \dots, (M_{n-1} \times A)$. Finally,
330 we concatenate the results to get the n -bit sum in the right hand side of Eq. (1).

331 Therefore, we can decompose the matrix-vector multiplication as follows

$$M_j \times A = \frac{1}{2^m} H_m(H_m(C_{\uparrow 1}^j) \times H_m(A)),$$

332 for $j \in \{0, 1, \dots, n-1\}$, where $C_{\uparrow 1}^j$ is the first column of M_j . As a result, the com-
 333 putation of $M_j \times A$ can be simplified to computing three Hadamard transforms with
 334 complexity of $3 \times m \times 2^m$ additions.

335 3 Key-Recovery Attacks on the ARADI Block Cipher

336 In this section, we present key recovery attacks against reduced-round ARADI. Our
 337 analysis covers attacks on up to 12 rounds of ARADI. Since the attacks on fewer than 10
 338 rounds follow a similar and relatively straightforward key recovery principle, we focus
 339 on providing detailed descriptions of the attacks on 10, 11, and 12 rounds only. In all
 340 of our attacks, we recover equivalent round keys of the alternative representation of
 341 ARADI described in Section 2.2.3 (also illustrated in Figure 1). To simplify the analysis,
 342 we omit the final linear layer operation of ARADI. This omission does not affect the
 343 attack’s effectiveness: if the last linear layer is included, the attack proceeds almost
 344 identically, with the only additional step being the application of the inverse of the
 345 last linear layer to each collected ciphertext before further processing.

346 Before delving into the specifics of each attack, we provide a high-level overview
 347 of the general attack strategy. The core strategy is to guess parts of the round keys,
 348 partially decrypt the ciphertexts, and check if the XOR sum of selected bits is zero.
 349 A critical aspect of the ARADI linear layers is that the partial decryption of one col-
 350 umn depends on three columns from the subsequent round. For instance, the partial
 351 decryption of the 0-th column at the 9-th round, denoted by $x_{\uparrow 0}^9$, depends on the 0-th,
 352 5-th, and 24-th columns from the 10-th round, denoted by $z_{\uparrow 0}^{10}$, $z_{\uparrow 5}^{10}$, and $z_{\uparrow 24}^{10}$, respec-
 353 tively. Consequently, to verify whether the XOR sum at any given bit of $x_{\uparrow 0}^9$ is zero, it
 354 suffices to guess only these three columns of the (equivalent) key state at the 10-th
 355 round, namely $u_{\uparrow 0}^{10}$, $u_{\uparrow 5}^{10}$, and $u_{\uparrow 24}^{10}$. We refer to these columns as *guessed columns*. By
 356 focusing on decrypting one column at a time and checking the XOR sum of its partial
 357 decryption, we effectively reduce the number of round key components that must be
 358 guessed at once. This approach is known as the partial sum technique [FKL⁺00]. This
 359 targeted approach streamlines the key recovery process and makes the attack more
 360 efficient.

361 **Remark 2.** *We note that for the attacks described in Section 3.1–3.3, the overall*
 362 *time complexity is largely dominated by the time needed to obtain the data. Hence, the*
 363 *reader may wonder why we optimize the attack steps. We emphasize that these attacks*
 364 *are used to illustrate and analyze the impact of various optimization techniques. Such*
 365 *optimizations become particularly relevant for the 11- and 12-round attacks presented*
 366 *in Section 3.4 and Section 3.5, respectively, where the analysis time is higher than the*
 367 *time needed for encrypting the data.*

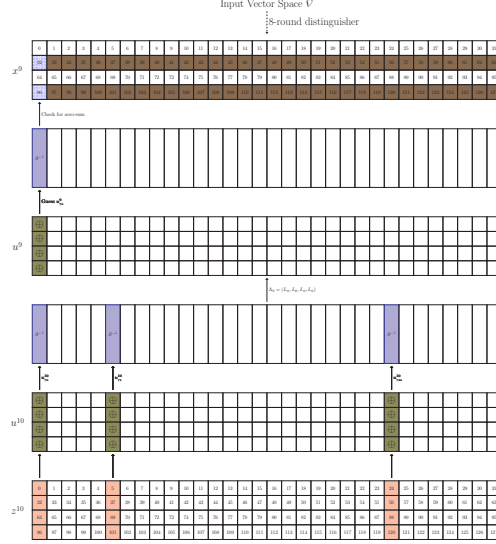


Fig. 2 Overview of the 10-round attack on ARADI using 8-round distinguisher

3.1 Attack on the 10-Round ARADI Cipher Using 8-Round Distinguisher

We begin by presenting a detailed description of the key recovery attack on 10 rounds of ARADI. This attack leverages the 8-round ZeroSum distinguisher from Table 4 to recover the round keys for the 9-th and 10-th rounds, denoted as $u^9 = \Lambda_9(K^9)$ and $u^{10} = \Lambda_{10}(K^{10})$, respectively.³

An overview of the attack is provided in Figure 2. The cube dimension used in the distinguisher is 124, highlighted as ■ in the figure. After 8 rounds, two rows of the state x^9 , the input to the 9-th round, becomes *balanced*, as indicated by ■ in the same figure. By appending 2 additional rounds to the 8-round distinguisher, we extend the attack to cover 10 rounds of ARADI. The following sections describe the detailed steps of the attack along with the corresponding computational complexities, focusing on the ZeroSum at the 0-th column $x_{\uparrow 0}^9$. The attack methodology for other columns is essentially the same.

The naïve attack. A naïve attack approach is to guess partial key bits from u^9 and u^{10} to perform a two-round partial decryption, followed by a ZeroSum check. Specifically, to partially decrypt the column $x_{\uparrow 0}^9$, we need to guess a total of four columns (16 bits) of the subkey: $u_{\uparrow 0}^9$ of u^9 and $u_{\uparrow 0}^{10}$, $u_{\uparrow 5}^{10}$, $u_{\uparrow 24}^{10}$ of u^{10} . These guessed columns are highlighted as ■ in Figure 2. The attack begins by querying 2^{124} encryptions corresponding to vector space of dimension 124 as given in Table 4. Let the set of ciphertexts be $\mathcal{C} = \{c^0, c^1, \dots, c^{2^{124}-1}\}$. The naïve attack is detailed in Algorithm 1, where we exhaustively guess the 16 key bits for the columns $u_{\uparrow 0}^9$, $u_{\uparrow 0}^{10}$, $u_{\uparrow 5}^{10}$ and $u_{\uparrow 24}^{10}$. For each guess, we partially decrypt the ciphertexts and verify whether XOR sum at the

³We remind the reader that the ARADI rounds are indexed starting from 1.

391 1st ($x_{1,0}^9$) and 3rd bit ($x_{3,0}^9$) of the partially decrypted column $x_{\uparrow 0}^9$ is zero. Formally,
 392 we check if 1-st and 3-rd bits of

$$\bigoplus_{c \in \mathcal{C}} x_{\uparrow 0}^9 = \bigoplus_{c \in \mathcal{C}} S^{-1}(u_{\uparrow 0}^9 \oplus S^{-1}(u_{\uparrow 0}^{10} \oplus c_{\uparrow 0}) \oplus S^{-1}(u_{\uparrow 5}^{10} \oplus c_{\uparrow 5}) \oplus S^{-1}(u_{\uparrow 24}^{10} \oplus c_{\uparrow 24})) \quad (2)$$

393 are zero, where S^{-1} denotes the inverse SBox operation of ARADI. This results in a
 394 2-bit filter, meaning that, from this ZeroSum check, we obtain 2 bits of information
 395 on the 16-bit subkey.

396 The data complexity of this attack is 2^{124} . The time complexity is $2^{124} + (2^{124} \times$
 397 $2^{16}) \approx 2^{140}$. We also, need to store the ciphertexts in the memory. In terms of memory,
 398 storing all ciphertexts requires $2^{124} \times 128 = 2^{131}$ bits. Note that, a similar attack on
 399 10-round ARADI was proposed in [BRRT25] with a complexity of 2^{192} . However, thanks
 400 to the equivalent representation of ARADI, while the attack in [BRRT25] required
 401 guessing 48 bits over two rounds, this attack requires guessing only 16. Consequently,
 402 significantly reducing the overall complexity of the attack.

Algorithm 1 Naïve algorithm for 10-round key recovery.

```

1: Input: A  $2^{124}$  size set of ciphertexts,  $\mathcal{C}$ .
2: for all  $(u_{\uparrow 0}^9, u_{\uparrow 0}^{10}, u_{\uparrow 5}^{10}, u_{\uparrow 24}^{10}) \in \{0, 1\}^{16}$  do
3:   sum = 0
4:   for all  $c \in \mathcal{C}$  do
5:     sum = sum  $\oplus S^{-1}(u_{\uparrow 0}^9 \oplus S^{-1}(u_{\uparrow 0}^{10} \oplus c_{\uparrow 0}) \oplus S^{-1}(u_{\uparrow 5}^{10} \oplus c_{\uparrow 5}) \oplus S^{-1}(u_{\uparrow 24}^{10} \oplus c_{\uparrow 24}))$ 
6:   if (sum1  $\neq 0$ ) or (sum3  $\neq 0$ ) then
7:      $(u_{\uparrow 0}^9, u_{\uparrow 0}^{10}, u_{\uparrow 5}^{10}, u_{\uparrow 24}^{10})$  is not a valid key candidate

```

3.1.1 Improved attack

403
 404 Having discussed the general framework of the FHT-based attack in Section 2.4, we
 405 now turn to the specific case of the FHT-based attack on 10 rounds of ARADI. Several
 406 optimizations can be applied to enhance the naïve attack using FHT-based tech-
 407 niques discussed in Section 2.4. First of all, we prepare a binary array A as discussed
 408 in Section 2.4. Note that the right-hand side of Equation (2) depends solely on the
 409 ciphertext columns $\{0, 5, 24\}$ of the ciphertext. This implies that if two ciphertexts
 410 share the same values in these three columns, their combined contribution to the XOR
 411 sum in Equation (2) cancels out independent of the key guesses. This is a well-known
 412 technique and used in several other attacks [FKL⁺00, TA14, DGK⁺24]. Exploiting
 413 this property, instead of storing all ciphertexts explicitly, we construct an array A with
 414 2^{12} indices, where each index corresponds to the parity of number of occurrences of a
 415 particular value in the list of ciphertexts. Formally, the entries of A are defined as:

$$A[i] = \begin{cases} 1, & \text{if } \forall c \in \mathcal{C}, c_{\uparrow 0} || c_{\uparrow 5} || c_{\uparrow 24} = i \text{ holds \textcolor{red}{odd} number of times} \\ 0, & \text{if } \forall c \in \mathcal{C}, c_{\uparrow 0} || c_{\uparrow 5} || c_{\uparrow 24} = i \text{ holds \textcolor{blue}{even} number of times} . \end{cases}$$

416 Suppose that the subkey $u_{\uparrow 0}^9$ has already been guessed, and let the guessed value
 417 be a . For each such guess, the expression in Equation (2) becomes:

$$\text{sum}^a[k_0||k_1||k_2] = \bigoplus_{\{c=c_0||c_1||c_2:A[c]=1\}} S^{-1}(a \oplus S^{-1}(k_0 \oplus c_0) \oplus S^{-1}(k_1 \oplus c_1) \oplus S^{-1}(k_2 \oplus c_2)).$$

418 As discussed in Section 2.4, we can replace these computation with a FHT. To apply
 419 the FHT-based method, we consider the convolution of two functions over 2^{12} inputs,
 420 defined as follows:

$$\begin{aligned} f^a(u_0||u_1||u_2) &= S^{-1}(a \oplus S^{-1}(x_0) \oplus S^{-1}(x_1) \oplus S^{-1}(x_2)) \text{ and} \\ g(c_0||c_1||c_2) &= \begin{cases} 1, & \text{if } A[c_0||c_1||c_2] = 1 \\ 0, & \text{if } A[c_0||c_1||c_2] = 0. \end{cases} \end{aligned}$$

421 Then, the sum on the right-hand side of Equation (2) can be computed as the
 422 convolution of these two functions:

$$\begin{aligned} &\bigoplus_{c \in \mathcal{C}} S^{-1}(u_{\uparrow 0}^9 \oplus S^{-1}(u_{\uparrow 0}^{10} \oplus c_{\uparrow 0}) \oplus S^{-1}(u_{\uparrow 5}^{10} \oplus c_{\uparrow 5}) \oplus S^{-1}(u_{\uparrow 24}^{10} \oplus c_{\uparrow 24})) \\ &= \bigoplus_{c \in \{0,1\}^{12}} g(c) f^{u_{\uparrow 0}^9}(u_{\uparrow 0}^{10}||u_{\uparrow 5}^{10}||u_{\uparrow 24}^{10}, c) \\ &= \left((g * f^{u_{\uparrow 0}^9})(u_{\uparrow 0}^{10}||u_{\uparrow 5}^{10}||u_{\uparrow 24}^{10}) \right) \bmod 2. \end{aligned}$$

423 These convolutions can be efficiently evaluated using FHT, allowing us to simultane-
 424 ously compute all 2^{12} values of the sum.

425 The full FHT-based key-recovery attack on 10 rounds is detailed in Algorithm 2.
 426 It is important to note that since f is a vectorial Boolean function, we must apply
 427 the FHTs separately to each output coordinate j . As we check the ZeroSum for $j = 1$
 428 and $j = 3$, we apply two sets of FHTs. Also, the FHT of the vector A is independent
 429 of the choice of $u_{\uparrow 0}^9$. As a result, it only needs to be computed once and can be reused
 430 for all 16 possible values of $u_{\uparrow 0}^9$, as shown in line 3 of Algorithm 2.

431 The overall complexity of Algorithm 2 includes: $2^4 \times 2$ pointwise multiplications
 432 of two vectors of size 2^{12} , $2^4 \times 2 \times 2$ FHTs of size 2^{12} , and a single FHT of size 2^{12} on
 433 the vector A . Thus, the total computational cost is approximately:

$$(12 \times 2^{12}) + 2^4 \times 2 \times (2^{12} + 12 \times 2^{12} + 2^{12} + 12 \times 2^{12} + 2^{12}) \approx 2^{21.77}.$$

434 The memory complexity is equal to storing the vector A , i.e., 2^{12} integers.

435 An implementation of the FHT-based attack strategy on 6 rounds of ARADI,
 436 based on a 4-round ZeroSum distinguisher, is available at is given as **supplementary**
 437 **material**.⁴

⁴The code is available at https://github.com/ShibamCrS/zerosum_attack_aradi.git

Algorithm 2 FHT-based algorithm for 10-round key-recovery.

```
1: Input: Array  $A$  of integers such that the  $i$ -th value of  $A$  denotes the parity of the
   number of occurrences of  $i$  in the list of ciphertexts
2: Initialize arrays  $\text{sum}_j$  of size  $2^{12}$  for  $j \in \{0, 1, 2, 3\}$ 
3:  $A \leftarrow H_{12} \times A$  ▷ Computing the first FHT
4: for all  $u_{\uparrow 0}^9 \in \{0, 1\}^4$  do
5:   for all  $j \in \{1, 3\}$  do
6:     for all  $u \in \{0, 1\}^{12}$  do ▷ Computing first column of  $M_j^{u_{\uparrow 0}^8}$ 
7:        $\text{sum}_j[u] \leftarrow f_j^{u_{\uparrow 0}^9}(u, 0)$ 
8:      $\text{sum}_j \leftarrow H_{12} \times \text{sum}_j$  ▷ Computing the second FHT
9:     for all  $u \in \{0, 1\}^{12}$  do
10:       $\text{sum}_j[u] \leftarrow \frac{1}{2^{12}}(\text{sum}_j[u] \times A[u])$ 
11:      $\text{sum}_j \leftarrow H_{12} \times \text{sum}_j$  ▷ Computing the third FHT
12:   for all  $(u_{\uparrow 0}^{10}, u_{\uparrow 5}^{10}, u_{\uparrow 24}^{10}) \in \{0, 1\}^{12}$  do
13:      $u \leftarrow u_{\uparrow 0}^{10} || u_{\uparrow 5}^{10} || u_{\uparrow 24}^{10}$ 
14:     for all  $j \in \{1, 3\}$  do
15:       if  $\text{sum}_j[u] \bmod 2 \neq 0$  then
16:          $(u_{\uparrow 0}^9, u_{\uparrow 0}^{10}, u_{\uparrow 5}^{10}, u_{\uparrow 24}^{10})$  is not a valid key candidate
```

3.1.2 Master-Key Recovery

In our attack on 10 rounds of ARADI, we initially apply the **ZeroSum** condition by considering a cube of dimension 124, on a single column — specifically, the 0-th column $x_{\uparrow 0}^9$. With this cube the **ZeroSum** condition satisfies in two bits after 8 rounds. Thus, the probability that a randomly chosen 16-bit subkey satisfies the **ZeroSum** condition after 8 rounds is 2^{-2} . This effectively results in a 2-bit filter, meaning that, for each **ZeroSum** check, approximately $2^{14} = 2^{16} \times 2^{-2}$ key candidates remain unfiltered. This filtering effect was not considered in the previously proposed 10-round attack in [BRRT25], where a single cube with 2-bit filter was used. Thus, we can say that the attack presented in [BRRT25] recovers only 2-bit information of the key from a single **ZeroSum** check.

To uniquely recover the 16 bits of the target subkey, we apply our attack with multiple independent and distinct cubes, each of dimension 124. This ensures that the overlap among surviving key candidates across multiple cubes is minimal, ultimately allowing a unique key recovery. We discuss the procedure in the following remark.

Remark 3. Let us consider two independent cubes $\mathcal{V}_1$ and $\mathcal{V}_2$, each of dimension 124 as given in Table 4. These cubes each yield **ZeroSum** distinguishers on the 1-st and 3-rd bits, as these bits have algebraic degree at most 123. This provides us with a 4-bit filter in total, with 2 bits contributed by each cube. Furthermore, if we select these cubes such that their union $\mathcal{V}_1 \cup \mathcal{V}_2$ has dimension 125, we gain additional filtering. In addition to the 4-bit filtering on the 1-st and 3-rd bits (two bits for each of the two cubes), we also obtain **ZeroSum** on the 0-th and 2-nd bits by considering the cube of

Table 5 An example of various combinations of 124 or 116 dimensional cubes used in our attack and their corresponding filter size

Last four bits of x for 124-dimensional cubes or any fixed set of 4 bits out of the last 12 bits for 116-dimensional cubes	Filter Size (bits)
(0000, 1000)	6
(0000, 0001, 0010)	10
(0000, 0001, 0010, 0011)	16
(0000, 0001, 0010, 0100, 0011, 0110, 0101)	32
(0000, 0001, 0010, 0100, 0011, 0110, 0101, 0111)	40

dimension 125, as these bits have a degree at most 124. In total, a 125-dimension cube offers a 6-bit filter.

Remark 4. Now the question remains: how many 124-dimensional cubes are needed to generate so that we can get desired filtering? To answer this, consider how we generate various 124-dimensional cubes. Let V be a 124-dimensional cube. From V , we can construct another 124-dimensional affine space as $V_i = \{x_i \oplus v \mid v \in V\}$, where x_i is a 128-bit vector. According to Table 4, the vectors in V take all possible values in the first 124 bits while the last 4 bits remain constant. Thus, changing one of these last 4 bits is required for the necessary affine translation. We therefore consider each x_i such that the first 124 bits are 0, i.e., it is of the form $00\dots 0v_1v_2v_3v_4$. Consequently, the union of two such affine spaces V_i and V_j has dimension 125 if the Hamming weight of $x_i \oplus x_j$ is 1. According to our analysis, we can obtain a 6-bit filter with two 124-dimensional cubes, a 10-bit filter with three 124-dimensional cubes, a 16-bit filter with four 124-dimensional cubes, 32-bit filter with seven 124-dimensional cubes, and a 40-bit filter with eight 124-dimensional cubes. Similar analysis can be done for 116-dimensional cubes as well where each vectors take all possible values in the first 116 bits while the last 12 bits remain constant as shown in Table 4. In this case, we can change any 4 bits of the last 12 bits for the necessary affine translation. We provide examples of such x_i vectors and their corresponding filter sizes in Table 5.

According to the above discussion in Remark 3 and Remark 4, if we chose four 124 dimensional cubes, we achieve a filtering power that is sufficient to recover the 16 bits of the target subkey. Consequently, the total data complexity is $4 \times 2^{124} = 2^{126}$, and the time complexity is $4 \times 2^{21.77} = 2^{23.35}$.

To extend this to the recovery of the full 256-bit master key, we focus on recovering the complete round keys for rounds 9 and 10. This involves repeating the above attack across all 32 columns — i.e., applying the ZeroSum test to each column from $x_{\uparrow 0}^9$ to $x_{\uparrow 31}^9$. As a result, the total time complexity becomes $2^{126} + 32 \times 2^{23.35} \approx 2^{126}$. Additionally, since the same data can be reused for ZeroSum checks across different columns, the overall data complexity remains 2^{126} . The memory complexity is determined by storing one vector A of size 2^{12} for each attack and each cube, resulting in a total of 2^{17} integers.

Remark 5. Note that recovering the full round keys for the 9-th and 10-th rounds reveals 192 bits of the master key, since two 32-bit words of the u^{10} can be derived from u^9 . This relation holds for any pair of consecutive rounds $(i, i+1)$ where i is odd. As a result, at the end to recover the full 256-bit master key, we still need to brute force 64 bits. On the other hand, this reduces the search space for the subkey $u^i || u^{i+1}$ by 64 bits when i is odd. This observation was also made in [BRRT25], although the authors claimed that it holds for all values of i . However, the dependency does not hold when i is even. In fact, if i is even, then knowing the full round keys for rounds i and $i+1$ is sufficient to uniquely determine the master key via the inverse key schedule, without additional brute-force. We leverage this insight in our later attacks by specifically targeting round keys for even indices i , enabling full master key recovery once u^i and u^{i+1} are known.

A different approach, which allows the use of two cubes (rather than four as mentioned before), is to exploit key relations and operate on different columns in an order that maximizes the filtering. To illustrate, consider the initial ZeroSum check on the 0-th column using two 124 dimensional cubes. This results in approximately 2^{10} candidate values for the 16-bit subkey as we have a 6-bit filter, which we store in a list denoted as $\mathcal{L}$. Next, we apply the ZeroSum condition to a second column, selected strategically to maximize the overlap between the required guessed key bits and those already guessed in the previous step. In this case, we choose the 19-th column. The partial decryption of $x_{\uparrow 19}^9$ depends on the subkeys $u_{\uparrow 5}^{10}$, $u_{\uparrow 19}^{10}$, and $u_{\uparrow 24}^{10}$. Notably, $u_{\uparrow 5}^{10}$ and $u_{\uparrow 24}^{10}$ were already guessed when handling the 0-th column, resulting in an 8-bit overlap.

This overlap acts as an effective 8-bit filter, narrowing the candidate space for the new subkeys. Specifically, for each value of $u_{\uparrow 5}^{10} || u_{\uparrow 24}^{10}$ in $\mathcal{L}$, we expect 2^2 suggestion on average for $u_{\uparrow 19}^9 || u_{\uparrow 19}^{10}$ while we check ZeroSum on the 19-th column using 2 cubes. To efficiently combine these results and update the list $\mathcal{L}$ with the new suggestions from the ZeroSum check on the 19-th column, we use a hash table. We transform the list $\mathcal{L}$ into a hash table, where each entry is keyed by the 8-bit value $u_{\uparrow 5}^{10} || u_{\uparrow 24}^{10}$ of the subkeys present in $\mathcal{L}$ i.e., the overlapping portion of the subkey. For each entry in $\mathcal{L}$, the associated value in the hash table is the corresponding 8-bit partial subkey $u_{\uparrow 0}^9 || u_{\uparrow 0}^{10}$. As we apply the ZeroSum check on the 19-th column, we generate new candidate pairs $(u_{\uparrow 5}^{10} || u_{\uparrow 24}^{10}, u_{\uparrow 19}^9 || u_{\uparrow 19}^{10})$. For each such pair, we query the hash table using the 8-bit key $u_{\uparrow 5}^{10} || u_{\uparrow 24}^{10}$ and we merge the old values $u_{\uparrow 0}^9 || u_{\uparrow 0}^{10}$ with the new ones $u_{\uparrow 19}^9 || u_{\uparrow 19}^{10}$ to form a longer partial subkey candidate. The updated list now contains approximately 2^{12} candidates for a 24-bit subkey comprising: $u_{\uparrow 0}^9 || u_{\uparrow 19}^9 || u_{\uparrow 0}^{10} || u_{\uparrow 5}^{10} || u_{\uparrow 19}^{10} || u_{\uparrow 24}^{10}$. By carefully selecting subsequent columns with maximal key-bit overlap with previously guessed subkeys, we are able to add more key columns while avoid exponential growth in the number of key candidates.

We repeat this process across multiple columns, progressively refining the candidate key list, until a sufficient number of key bits have been recovered such that performing an exhaustive search over the remaining bits becomes more efficient than performing additional ZeroSum attacks. For instance, we perform the ZeroSum attack on the following sequence of 30 columns (in the specified order): 0, 19, 11, 30, 6, 25, 1, 20, 12, 31, 7, 26, 2, 21, 13, 16, 8, 27, 3, 22, 14, 17, 9, 28, 4, 23, 15, 18, 10, 5, as listed in the

Table 6 Progressive ZeroSum checks in 10-round attack using 8-round distinguisher

Step	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Selected Column $x_{\uparrow?}^8$	0	19	11	30	6	25	1	20	12	31	7	26	2	21	13
$\log_2(\mathcal{L})$	10	12	14	16	18	20	22	24	26	28	30	31	33	34	36
Candidate Size (bits)	16	24	32	40	48	56	64	72	80	88	96	103	111	118	126

Step	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Selected Column $x_{\uparrow?}^8$	16	8	27	3	22	14	17	9	28	4	23	15	18	10	5
$\log_2(\mathcal{L})$	37	38	40	42	42	44	42	39	36	32	28	24	20	16	12
Candidate Size (bits)	133	140	148	156	162	170	174	177	180	182	184	186	188	190	192

second row of Table 6. At each step, the attack yields candidate values for a 16-bit subkey, with the size of the candidate list $\mathcal{L}$ provided in the third row.

Importantly, due to the linear key-schedule, the 1-st and 3-rd rows of u^{10} are linearly dependent on u^9 , as previously discussed in Remark 5. Exploiting these linear relations allows us to filter additional key bits. In particular, after a certain number of steps, newly considered key bits can be derived from linear combinations of key bits obtained in earlier steps. In such cases, these dependent key bits do not need to be guessed explicitly; instead, we check the ZeroSum property only for those entries whose dependent key bits satisfy the corresponding linear relations. To ensure that we only count independent key information, we analyze the rank of the relevant linear systems. We construct these systems symbolically by expressing each key bit of u^9 and u^{10} as a linear combination of the master-key bits, and then verify the rank using SageMath.⁵ We performed analogous checks for the other attacks in the paper as well; the code for all verifications is provided in the **supplementary material**.⁶

This effect is reflected in Table 6. The third row reports the size of the candidate list $\mathcal{L}$ at each step, which depends on whether the newly considered key bits are linearly dependent on previously introduced bits. Typically, the list size increases by a factor of 4 per step, as expected. The first step where a dependency appears is the 11-th step, where the increase drops to a factor of 2. Consequently, the fourth row of Table 6 reports the bit length of each element, that is, the number of independent key bits in $\mathcal{L}$, corresponding to the accumulated key material at that step.

In summary, we apply 30 ZeroSum attacks, each using two 124 dimensional cubes (total data: 2^{125}). These correspond to a total of 120 bits of u^9 and 128 bits of u^{10} and the final candidate list $\mathcal{L}$ contains approximately 2^{12} elements. To complete the key recovery, we can brute-force the remaining 8 bits of u^9 across the 2^{12} candidates in $\mathcal{L}$, resulting in 2^{20} key combinations. The time complexity of this part of the attack is $2^{22.77}$. Moreover, as discussed in Remark 5, we can derive 192 bits of the master key from the known values of u^9 and u^{10} . Thus, we need to recover 64 bits through exhaustive search, i.e., the total time complexity becomes $2^{125} + 2^{64+20} + (30 \times 2^{22.77}) \approx 2^{125}$. The memory complexity is corresponding to storing the vector A of size 2^{12} integers for each of the 30 attacks and each of the 2 cubes. Furthermore, to store the

⁵SageMath is an open-source mathematics software system, freely available at <https://www.sagemath.org>.

⁶The code is available at https://github.com/ShibamCrS/zerosum_attack_aradi.git

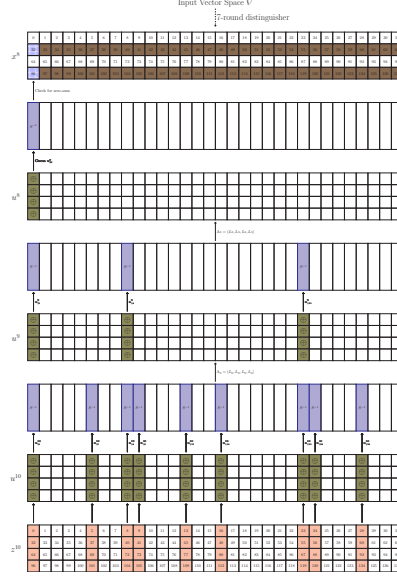


Fig. 3 Overview of the 10-round attack on ARADI using 7-round distinguisher

key suggestions, we need to store at most 192×2^{44} bits. Thus, the total memory complexity is upper-bounded by 2^{76} bits.

3.2 A 10-Round Key-Recovery Using 7-Round Distinguisher

Previously, we discussed a 10-round key-recovery attack based on an 8-round distinguisher. However, as observed, the complexity of those attacks is dominated by the data collection cost of the distinguisher. The primary motivation behind the attack presented here is to reduce that cost by using a more efficient 7-round distinguisher, which has a lower data complexity of 2^{116} chosen plaintexts. Applying a 3R-attack, i.e., adding 3 rounds for key-recovery, we construct a 10-round attack.

The main attack procedure is the same as the 8-round attack. We start with encrypting a vector space of dimension 116 as shown in Table 4. After 7 rounds of encryption, all bits in the 1-st ($x_{1 \rightarrow}^8$) and 3-rd ($x_{3 \rightarrow}^8$) rows of the output are balanced (i.e., their XOR sum is zero). Therefore, we partially decrypt the ciphertexts through three rounds and check whether the XOR sum equals to zero at the 1-st and 3-rd rows.

For example, the attack targeting the 0-th column is illustrated in Figure 3. In this case, the computation of the 0-th column in the 8-th round depends on columns 0, 8, 23 from the 9-th round. In turn, the computation of columns 0, 8, 23 in the 9-th round depends on the following 9 columns from the 10-th round: $\{0, 5, 8, 9, 13, 16, 23, 24, 28\}$. This is shown in Figure 3. Following the same methodology as before, we start by encrypting a cube of dimension 120. Let the resulting set of ciphertexts be denoted

586 by $\mathcal{C} = \{c^0, c^1, \dots, c^{2^{116}-1}\}$. Our goal is then to compute the following sum:

$$\bigoplus_{c \in \mathcal{C}} S^{-1} \begin{pmatrix} u_{\uparrow 0}^8 \oplus \\ S^{-1}(u_{\uparrow 0}^9 \oplus S^{-1}(u_{\uparrow 0}^{10} \oplus c_{\uparrow 0})) \oplus S^{-1}(u_{\uparrow 5}^{10} \oplus c_{\uparrow 5}) \oplus S^{-1}(u_{\uparrow 24}^{10} \oplus c_{\uparrow 24})) \oplus \\ S^{-1}(u_{\uparrow 8}^9 \oplus S^{-1}(u_{\uparrow 8}^{10} \oplus c_{\uparrow 8})) \oplus S^{-1}(u_{\uparrow 13}^{10} \oplus c_{\uparrow 13}) \oplus S^{-1}(u_{\uparrow 16}^{10} \oplus c_{\uparrow 16})) \oplus \\ S^{-1}(u_{\uparrow 23}^9 \oplus S^{-1}(u_{\uparrow 23}^{10} \oplus c_{\uparrow 23})) \oplus S^{-1}(u_{\uparrow 28}^9 \oplus c_{\uparrow 28})) \end{pmatrix} \quad (3)$$

587 One way to proceed in this case is, first, we guess a 16-bit key consisting of
 588 $(u_{\uparrow 0}^8, u_{\uparrow 0}^9, u_{\uparrow 8}^9, u_{\uparrow 23}^9)$. Then, we recover the remaining keys using the FHT-based attack
 589 described above where the size of FHT is 2^{36} , which is sub-optimal. To overcome this
 590 challenge, we adopt a more efficient, multi-step strategy based on the partial sum
 591 technique.

592 3.2.1 An Efficient Key Recovery using Partial Sums and FHT

593 The *partial sum technique*, first introduced by Ferguson et al. [FKL⁺00], is a pow-
 594 erful method that significantly reduces the complexity of computing large XOR sums
 595 by breaking the problem into smaller, sequential steps. This approach was recently
 596 enhanced by Dunkelman et al. [DGK⁺24], who showed that several partial sum steps
 597 can be efficiently replaced by computing convolutions of carefully chosen functions.
 598 Our attack adapts this convolution-based methodology to the specific structure of the
 599 ARADI cipher and several convolutions is computed simultaneously using FHT as dis-
 600 cussed in Section 2.4. The main non trivial task is to identify which parts of the XOR
 601 sums are computed together in each step so that we can compute them efficiently
 602 using FHT. In our attack we target to efficiently compute a relationship involving the
 603 intermediate variables a_0, a_1, a_2 , and b , which are defined as follows:

$$\begin{aligned} a_0 &= S^{-1}(u_{\uparrow 0}^{10} \oplus c_{\uparrow 0}) \oplus S^{-1}(u_{\uparrow 5}^{10} \oplus c_{\uparrow 5}) \oplus S^{-1}(u_{\uparrow 24}^{10} \oplus c_{\uparrow 24}) \\ a_1 &= S^{-1}(u_{\uparrow 8}^{10} \oplus c_{\uparrow 8}) \oplus S^{-1}(u_{\uparrow 13}^{10} \oplus c_{\uparrow 13}) \oplus S^{-1}(u_{\uparrow 16}^{10} \oplus c_{\uparrow 16}) \\ a_2 &= S^{-1}(u_{\uparrow 9}^{10} \oplus c_{\uparrow 9}) \oplus S^{-1}(u_{\uparrow 23}^{10} \oplus c_{\uparrow 23}) \oplus S^{-1}(u_{\uparrow 28}^9 \oplus c_{\uparrow 28}) \\ b &= S^{-1}(u_{\uparrow 0}^9 \oplus a_0) \oplus S^{-1}(u_{\uparrow 8}^9 \oplus a_1) \oplus S^{-1}(u_{\uparrow 23}^9 \oplus a_2) \end{aligned} \quad (4)$$

604 Our process begins by storing the parity of the collected ciphertexts $\mathcal{C}$ in a binary
 605 array A of size 2^{36} :

$$A[i] = \begin{cases} 1, & \text{if } |c \in \mathcal{C} : c_{\uparrow 0, 5, 24, 8, 13, 16, 9, 23, 28} = i| \text{ is odd} \\ 0, & \text{otherwise} \end{cases}$$

606 To simplify notation throughout the attack, we also define two helper functions: an
 607 indicator function I_γ and a function g_χ to convert a binary array into a function
 608 suitable for convolution.

$$g_\chi(i) := \chi[i] \text{ and } I_\gamma(x_0, x_1, x_2) := \begin{cases} 1, & \text{if } S^{-1}(x_0) \oplus S^{-1}(x_1) \oplus S^{-1}(x_2) = \gamma \\ 0, & \text{otherwise} \end{cases}$$

609 The complete procedure is outlined in Algorithm 3. It consists of four main reduction
 610 steps followed by a final checking of ZeroSum, which we now describe in detail. The
 611 core of our attack is an iterative process that progressively reduces the size of the data
 612 array A by isolating the variables from Equation (4). Each step uses an FHT-based
 613 method to compute convolutions.

614 **Step 1: Reducing A to A_1 .** Our first goal is to reduce the 2^{36} size bit array A to a
 615 smaller array A_1 . The key observation is that if two ciphertexts yield the same value
 616 for the tuple $(a_0, c_{\uparrow 8,13,16,9,23,28})$, their contributions to the overall XOR sum cancel out.
 617 We can therefore sum over the remaining ciphertext bits and store only the parity of
 618 occurrences for each value of this tuple $(a_0, c_{\uparrow 8,13,16,9,23,28})$. This reduction is achieved
 619 by guessing the key bits $u_{\uparrow 0,5,24}^{10}$ and computing a convolution between g_A and I_{a_0} . For
 620 each guess of $u_{\uparrow 0,5,24}^{10}$, we compute and store:

$$A_1[u_{\uparrow 0,5,24}^{10}][a_0, c_{\uparrow 8,13,16,9,23,28}] = ((g_A * I_{a_0})(u_{\uparrow 0,5,24}^{10})) \bmod 2,$$

621 where the convolution is defined as:

$$(g_A * I_{a_0})(u_{\uparrow 0,5,24}^{10}) = \sum_{c_{\uparrow 0,5,24}} g_A[c_{\uparrow 0,5,24,8,13,16,9,23,28}] \cdot I_{a_0}(u_{\uparrow 0}^{10} \oplus c_{\uparrow 0}, u_{\uparrow 10} 5 \oplus c_{\uparrow 5}, u_{\uparrow 24}^{10} \oplus c_{\uparrow 24}).$$

622 As discussed earlier in Section 2.4, using the FHT, we can perform this for all 2^{12} key
 623 guesses simultaneously. This corresponds to lines 3 to 5 in Algorithm 3.

624 **Steps 2-4: Iterative Reductions.** The subsequent steps follow the same pattern,
 625 each time further reducing the array size. Thus, we skip the details and only provide
 626 the functions of which the convolutions are computed.

627 *Step 2:* The array A_1 is reduced to an array A_2 that stores the parity of occurrences
 628 for each value of $(a_1, a_0, c_{\uparrow 9,23,28})$. This is done for all 2^{12} guesses of $u_{\uparrow 8,13,16}^{10}$ via
 629 the FHT-based technique of computing convolutions $(g_{A_1} * I_{a_1})(u_{\uparrow 8,13,16}^{10})$. This is
 630 done in lines 8 to 10 of Algorithm 3.

631 *Step 3:* Similarly, A_2 is reduced to A_3 by processing bits related to a_2 and storing the
 632 parity of occurrences for each value of (a_2, a_1, a_0) for each $u_{\uparrow 9,23,28}^{10}$. This is done
 633 by computing $(g_{A_2} * I_{a_2})$ in lines 13 to 15 of Algorithm 3.

634 *Step 4:* Finally, A_3 is reduced to the array B which stores the parity of occurrences of
 635 b for each value of the 9th round subkey bits $u_{\uparrow 0,8,23}^9$. This is done by computing
 636 $(g_{A_3} * I_b)$ in lines 18 to 20 of Algorithm 3.

Final step. After the four reduction stages, we have the array B . The final step is
 to guess the remaining subkey $u_{\uparrow 0}^8$ and compute the following 4-bit sum directly for
 each guess:

$$\bigoplus_b B[u_{\uparrow 0}^9, u_{\uparrow 8}^9, u_{\uparrow 23}^9][b] \cdot S^{-1}(u_{\uparrow 0}^8 \oplus b)$$

637 For this last computation, we do not use FHT. As discussed in Section 2.4, computing
 638 a 4-bit sum via FHT requires 4×3 hadamard transforms of size 2^4 , leading to a
 639 complexity of 3×2^8 . A direct computation, in contrast, only requires 2^8 operations
 640 and is therefore more efficient.

641 **Complexity of a Single Reduction Step.** Each of the first four steps involves
 642 computing convolutions. As detailed in Section 2.4, this requires three FHTs and one
 643 pointwise multiplication. A crucial optimization is that the Hadamard transform of
 644 the indicator functions I_{a_0} , I_{a_1} , I_{a_2} and I_b does not depend on any guessed key bits.
 645 Therefore, these transformations can be pre-computed once and reused in every step,
 646 as we have done in Algorithm 2 for the data array A . This leaves two FHTs of size
 647 2^{12} and one pointwise multiplication per step. The complexity of a single step is thus
 648 $2^{12} + (2 \times 12 \times 2^{12}) \approx 2^{16.64}$ integer additions/subtractions.

Total Complexity. The total time complexity of the analysis is the sum of all parts:
 the pre-computation, the four reduction steps (with increasing data dependencies),
 and the final step. This sums to:

$$\underbrace{12 \cdot 2^{12}}_{\text{Pre-comp.}} + \underbrace{(2^{28} + 2^{32} + 2^{36} + 2^{40}) \cdot 2^{16.64}}_{\text{Steps 1-4}} + \underbrace{2^{48} \cdot 2^8}_{\text{Final Step}} \approx 2^{57.4}.$$

649 The memory complexity is dominated by the size of initial integer array A , which is of
 650 size 2^{41} bits.

651 3.2.2 Packing Multiple FHTs Together

652 To further enhance the efficiency of our attack, we adapt the *packing technique*, from
 653 Dunkelman et al. [DGK⁺24]. The core idea is to leverage the wide-word arithmetic
 654 capabilities of modern CPUs (e.g., 64-bit operations) to compute multiple small FHTs
 655 in parallel. This is achieved by *packing* several independent computations into a single
 656 `uint64_t` integer, treating it as a set of distinct bit-slices. Although we have not
 657 implemented our full attack due to its high overall complexity, we present this analysis
 658 to demonstrate the expected gain.

659 We exemplify this approach by showing how it can be applied to **Step 1** of our
 660 attack; the optimization for other steps follows a similar logic. Recall that in **Step 1**,
 661 for a fixed tuple $(a_0, c_{\uparrow 8, 13, 16, 9, 23, 28})$, we compute the convolution $(g_A * I_{a_0})$. The key
 662 insight is to split the summation space. Let g_A^j denotes the function derived from A
 663 where the most significant m bits of $c_{\uparrow 8, 13, 16, 9, 23, 28}$ are fixed to j . Thus, for each of the
 664 values in the remaining bits of $c_{\uparrow 8, 13, 16, 9, 23, 28}$, denoted by c^ℓ we have to compute the
 665 following 2^m convolutions:

$$\left((g_A^0 * I_{a_0})(u_{\uparrow 0, 5, 24}^{10}), (g_A^1 * I_{a_0})(u_{\uparrow 0, 5, 24}^{10}), \dots, (g_A^{2^m-1} * I_{a_0})(u_{\uparrow 0, 5, 24}^{10}) \right), \text{ where}$$

$$(g_A^j * I_{a_0})(u_{\uparrow 0, 5, 24}^{10}) = \sum_{c_{\uparrow 0, 5, 24}} g_A[j \| c^\ell] \cdot I_{a_0}(u_{\uparrow 0}^{10} \oplus c_{\uparrow 0}, u_{\uparrow 5}^{10} \oplus c_{\uparrow 5}, u_{\uparrow 24}^{10} \oplus c_{\uparrow 24}).$$

Algorithm 3 FHT-based attack on 10-round using 7-round Distinguisher.

```

1: Input: Array  $A$  of integers consisting of the parity
2: Initialize an empty 2D bit-array  $A_1$  of size  $2^{12} \times 2^{28}$ 
3: for all  $a_0, c_{\uparrow 8,13,16,9,23,28}$  do
4:   for all  $u_{\uparrow 0,5,24}^{10}$  do
5:      $A_1[u_{\uparrow 0,5,24}^{10}][a_0, c_{\uparrow 8,13,16,9,23,28}] \leftarrow$ 
       
$$\bigoplus_{c_{\uparrow 0,5,24}} A[c_{\uparrow 0,5,24,8,13,16,9,23,28}] \cdot I_{a_0}(u_{\uparrow 0}^{10} \oplus c_{\uparrow 0}, u_{\uparrow 5}^{10} \oplus c_{\uparrow 5}, u_{\uparrow 24}^{10} \oplus c_{\uparrow 24})$$

6:   for all  $u_{\uparrow 0,5,24}^{10}$  do
7:     Initialize an empty 2D bit-array  $A_2$  of size  $2^{12} \times 2^{20}$ 
8:     for all  $a_1, a_0, c_{\uparrow 9,23,28}$  do
9:       for all  $u_{\uparrow 8,13,16}^{10}$  do
10:         $A_2[u_{\uparrow 8,13,16}^{10}][a_1, a_0, c_{\uparrow 9,23,28}] \leftarrow$ 
          
$$\bigoplus_{c_{\uparrow 8,13,16}} A_1[u_{\uparrow 0,5,24}^{10}][a_0, c_{\uparrow 8,13,16,9,23,28}] \cdot I_{a_1}(u_{\uparrow 8}^{10} \oplus c_{\uparrow 8}, u_{\uparrow 13}^{10} \oplus c_{\uparrow 13}, u_{\uparrow 16}^{10} \oplus c_{\uparrow 16})$$

11:        for all  $u_{\uparrow 8,13,16}^{10}$  do
12:          Initialize an empty 2D bit-array  $A_3$  of size  $2^{12} \times 2^{12}$ 
13:          for all  $a_2, a_1, a_0$  do
14:            for all  $u_{\uparrow 9,23,28}^{10}$  do
15:               $A_3[u_{\uparrow 9,23,28}^{10}][a_2, a_1, a_0] \leftarrow$ 
                
$$\bigoplus_{c_{\uparrow 9,23,28}} A_2[u_{\uparrow 8,13,16}^{10}][a_1, a_0, c_{\uparrow 9,23,28}] \cdot I_{a_2}(u_{\uparrow 9}^{10} \oplus c_{\uparrow 9}, u_{\uparrow 23}^{10} \oplus c_{\uparrow 23}, u_{\uparrow 28}^{10} \oplus c_{\uparrow 28})$$

16:              for all  $u_{\uparrow 9,23,28}^{10}$  do
17:                Initialize an empty 2D bit-array  $B$  of size  $2^{12} \times 2^4$ 
18:                for all  $b$  do
19:                  for all  $u_{\uparrow 0}^9, u_{\uparrow 8}^9, u_{\uparrow 23}^9$  do
20:                     $B[u_{\uparrow 0,8,23}^9][b] \leftarrow$ 
                      
$$\bigoplus_{a_2, a_1, a_0} A_3[u_{\uparrow 9,23,28}^{10}][a_2, a_1, a_0] \cdot I_b(u_{\uparrow 0}^9 \oplus a_2, u_{\uparrow 8}^9 \oplus a_1, u_{\uparrow 23}^9 \oplus a_0)$$

21:                    for all  $u_{\uparrow 0,8,23}^9$  do
22:                      Initialize an empty integer-array  $S$  of size  $2^4$ 
23:                      for all  $u_{\uparrow 0}^8$  do
24:                         $S[u_{\uparrow 0}^8] \leftarrow \bigoplus_b B[u_{\uparrow 0}^9, u_{\uparrow 8}^9, u_{\uparrow 23}^9][b] \cdot S^{-1}(u_{\uparrow 0}^8 \oplus b)$ 
25:                        for all  $u_{\uparrow 0}^8$  do
26:                          if  $S[u_{\uparrow 0}^8] \neq 0$  then Discard  $(u_{\uparrow 0}^8, u_{\uparrow 0,8,23}^9, u_{\uparrow 0,5,24,8,13,16,9,23,28}^{10})$ 

```

666 This reformulation shows that the original computation can be decomposed into 2^m
 667 independent FHTs. The code idea is that these 2^m FHTs can be computed in parallel
 668 within a single integer by packing them as

$$\begin{aligned}
 & \sum_{c_{\uparrow 0,5,24}} \left(\sum_{j=0}^{2^m} 2^{sj} g_A[j][c^\ell] \right) \cdot I_{a_0}(u_{\uparrow 0}^{10} \oplus c_{\uparrow 0}, u_{\uparrow 5}^{10} \oplus c_{\uparrow 5}, u_{\uparrow 24}^{10} \oplus c_{\uparrow 24}) \\
 &= \sum_{j=0}^{2^m-1} 2^{sj} \sum_{c_{\uparrow 0,5,24}} g_A[j][c^\ell] \cdot I_{a_0}(u_{\uparrow 0}^{10} \oplus c_{\uparrow 0}, u_{\uparrow 10}^{10} \oplus c_{\uparrow 5}, u_{\uparrow 24}^{10} \oplus c_{\uparrow 24}) \\
 &= \sum_{j=0}^{2^m-1} 2^{sj} \sum_{c_{\uparrow 0,5,24}} (g_A^j * I_{a_0})(u_{\uparrow 0,5,24}^{10}).
 \end{aligned}$$

669 We introduce a “separation unit” of s bits and compute the sum $\sum_{j=0}^{2^m-1} 2^{sj} (g_A^j * I_{a_0})$. If s is chosen large enough such that the result of each individual convolution
 670 does not exceed 2^s (preventing overflow between bit-slices), then the final result of
 671 each $(g_A^j * I_{a_0}) \bmod 2$ can be extracted from the sj -th bit of the final integer sum.
 672

673 The choice of the number of parallel computations (2^m) and the separation unit (s)
 674 is critical and depends on the target data type. Assuming 64-bit operations (`uint64_t`),
 675 we have experimentally verified that for FHTs of size 2^{12} , we can safely pack 4 FHTs
 676 in parallel by choosing a separation of $s = 16$ bits for each. This implementation is
 677 given in the **supplementary material**⁷ and can be similarly applied to **Steps 2-4**.

678 This technique also provides a significant optimization for the **Final Step**. Pre-
 679 viously, we used a direct summation because an FHT-based approach offered no
 680 performance benefit. However, with packing, this changes. Since the **final step**
 681 involves FHTs of size 2^4 , we can pack 16 such FHTs in parallel within a 64-bit word
 682 (using $s = 4$). Incorporating these speedups, the improved time complexity of the
 683 analysis is:

$$\underbrace{12 \cdot 2^{12} + (4 \cdot 2^4)}_{\text{Pre-comp.}} + \underbrace{(2^{28} + 2^{32} + 2^{36} + 2^{40}) \cdot 2^{-2} \cdot 2^{16.64}}_{\text{Steps 1-4}} + \underbrace{(2^{48} \cdot 2^{-4} \cdot 2^{5.04})}_{\text{Final Step}} \approx 2^{54.76}.$$

684 This is at least a $6\times$ faster than the attack described in Section 3.2.1. It is worth
 685 noting that this parallelization is a direct benefit of our multi-step strategy. A direct
 686 FHT of size 2^{36} would be too large for packing and parallelizing even two such
 687 transforms is not possible in a 64-bit word.

688 3.2.3 Unique Master-Key Recovery

689 Similar to the discussion in the case of previous attack in Section 3.1.2, here each
 690 ZeroSum check gives us a 2-bit filter. Consequently, we need to use about 26 cubes to

⁷The code is available at https://github.com/ShibamCrS/zerosum_attack_aradi.git

Table 7 Progressive ZeroSum checks in 10-round attack using 7-round distinguisher

Step	1	2	3	4	5	6	7	8	9	10	11	12	13
Selected Column $x_{\uparrow?}^8$	0	8	23	31	4	12	19	27	3	11	18	26	7
$\log_2(\mathcal{L})$	36	40	28	16	28	24	12	0	12	8	0	0	0
Candidate Size (bits)	52	72	76	80	108	120	124	128	156	168	172	176	204
Step	14	15	16	17	18	19	20	21	22	23	24	25	-
Selected Column $x_{\uparrow?}^8$	15	22	30	2	10	17	25	1	9	16	24	5	-
$\log_2(\mathcal{L})$	0	0	0	0	0	0	0	0	0	0	0	0	-
Candidate Size (bits)	215	219	221	246	253	255	255	256	256	256	256	256	-

recover 52 bits information about the key, which results in data complexity of $2^{120.7}$ chosen plaintexts.

To reduce the data complexity, we consider 4 cubes of dimension 116 with 16-bit filter, resulting in a data complexity of 2^{118} . Although the size of the filter is less than 52 bits, it is still sufficient to recover the key using the same strategy of combining information across different columns, as described in Section 3.1.2. To achieve this, we perform ZeroSum checks on 25 carefully selected columns (in the specified order), as listed in the second row of Table 7. These columns are chosen using the same strategy of maximizing overlap in subkey bits, as described earlier in Section 3.1.2. At each step, the attack yields candidate values for a 52-bit subkey, with the size of the candidate list $\mathcal{L}$ provided in the third row. Additionally, the fourth row shows the bit length of each element in $\mathcal{L}$, corresponding to the accumulated key material at that stage. Here also we use the linear structure of the key schedule as the round key u^{10} can be obtained linearly from u^8 and u^9 . After these steps, we expect to obtain a unique suggestions for 100 bits of u^8 and 108 bits are from u^9 . To complete the key recovery, we need the remaining 28 bits of u^8 and 20 bits of u^9 .

Hence, the time complexity of the analysis is $25 \times 4 \times 2^{57.4} + 2^{48} \approx 2^{64.03}$ without the packing technique. Using the packing technique, the time complexity is $25 \times 4 \times 2^{54.76} + 2^{48} \approx 2^{61.4}$. The memory complexity is corresponding to storing the vector A for each of the 25 attacks. Furthermore, to store the key suggestions, we need to store at most 72×2^{40} bits. Thus, the total memory complexity is upper-bounded by 2^{48} bits.

3.3 A 11-Round Key-Recovery Using 8-Round Distinguisher

The 11-round key-recovery attack on ARADI follows a strategy similar to the 10-round attack that employs a 7-round ZeroSum distinguisher, thus we skip low level details. In this case, we extend the attack by using an 8-round ZeroSum distinguisher given in Table 4 and appending three rounds of partial decryption, thereby targeting 11 rounds in total. As with the 10-round case, the complexity of recovering each column of the round key remains approximately $2^{57.4}$ or $2^{54.76}$ using the packing technique.

To recover the master key, we utilize 4 independent cubes of dimension 124 with total 16-bit filter, resulting in a total data complexity of 2^{126} . We focus on recovering the complete round keys for the 10-th and 11-th rounds. To recover the round keys of the 10-th and 11-th rounds, we apply ZeroSum checks across 12 selected columns, as

Table 8 Progressive ZeroSum checks in 11-round attack using 8-round distinguisher

Step	1	2	3	4	5	6	7	8	9
Selected Column $x_{\uparrow?}^9$	0	19	11	30	6	25	1	20	12
$\log_2(\mathcal{L})$	36	40	40	40	40	40	36	36	32
Candidate Size (Bits)	52	72	88	104	120	136	148	164	176
Step	10	11	12	13	14	15	16	17	-
Selected Column $x_{\uparrow?}^9$	31	7	26	2	21	13	16	8	-
$\log_2(\mathcal{L})$	31	23	20	10	0	0	0	0	-
Candidate Size (Bits)	191	199	212	218	222	225	228	230	-

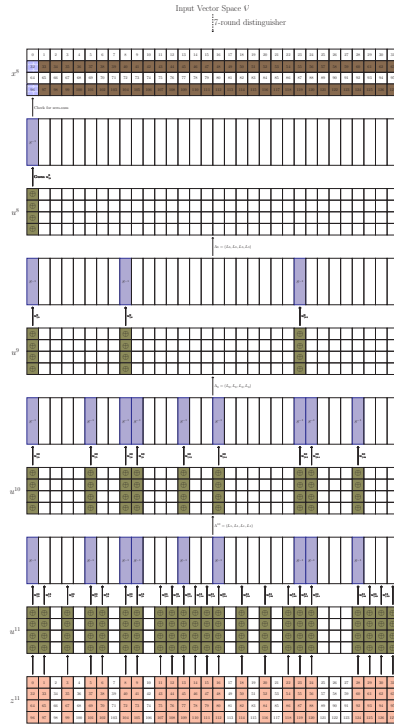


Fig. 4 Overview of the 11-round attack on ARADI using 7-round distinguisher

specified in Table 8. We also use the linear dependency round keys to filter key-bits as discussed in previous attacks. After performing these checks, we expect to obtain a unique candidates for 76 bits of u^{10} and 124 bits of u^{11} . So we need to recover remaining 52 bits of u^{10} and 4 bits of u^{11} . The total time complexity includes 16 ZeroSum-based attacks and a final brute-force over 2^{64} candidates: $17 \times 4 \times 2^{57.4} + 2^{56} \approx 2^{63.5}$ or $17 \times 4 \times 2^{54.76} + 2^{56} \approx 2^{60.9}$. To store the table $\mathcal{L}$, we need at most 2^{48} bits which is the upper bound on memory complexity.

3.4 A 11-Round Attack Using 7-Round Distinguisher

To reduce the data complexity of the 11-round attack presented in Section 3.3, we can utilize the 7-round ZeroSum distinguisher presented in Table 4 and extend it by 4 additional rounds to mount a key-recovery attack on 11 rounds of ARADI. The structure of the attack targeting the 0-th column is illustrated in Figure 4, where we compute the following sum:

$$\bigoplus_{c \in \mathcal{C}} S^{-1} \left(\begin{array}{l} u_{\uparrow 0}^8 \oplus \\ S^{-1} \left(\begin{array}{l} u_{\uparrow 0}^9 \oplus \\ S^{-1}(u_{\uparrow 0}^{10} \oplus S^{-1}(u_{\uparrow 0}^{11} \oplus c_{\uparrow 0}) \oplus S^{-1}(u_{\uparrow 6}^{11} \oplus c_{\uparrow 6}) \oplus S^{-1}(u_{\uparrow 23}^{11} \oplus c_{\uparrow 23})) \oplus \\ S^{-1}(u_{\uparrow 5}^{10} \oplus S^{-1}(u_{\uparrow 5}^{11} \oplus c_{\uparrow 5}) \oplus S^{-1}(u_{\uparrow 11}^{11} \oplus c_{\uparrow 11}) \oplus S^{-1}(u_{\uparrow 28}^{11} \oplus c_{\uparrow 28})) \oplus \\ S^{-1}(u_{\uparrow 24}^{10} \oplus S^{-1}(u_{\uparrow 13}^{11} \oplus c_{\uparrow 13}) \oplus S^{-1}(u_{\uparrow 24}^{11} \oplus c_{\uparrow 24}) \oplus S^{-1}(u_{\uparrow 30}^{11} \oplus c_{\uparrow 30})) \end{array} \right) \\ S^{-1} \left(\begin{array}{l} u_{\uparrow 8}^9 \oplus \\ S^{-1}(u_{\uparrow 8}^{10} \oplus S^{-1}(u_{\uparrow 8}^{11} \oplus c_{\uparrow 8}) \oplus S^{-1}(u_{\uparrow 14}^{11} \oplus c_{\uparrow 14}) \oplus S^{-1}(u_{\uparrow 31}^{11} \oplus c_{\uparrow 31})) \oplus \\ S^{-1}(u_{\uparrow 13}^{10} \oplus S^{-1}(u_{\uparrow 13}^{11} \oplus c_{\uparrow 13}) \oplus S^{-1}(u_{\uparrow 13}^{11} \oplus c_{\uparrow 13}) \oplus S^{-1}(u_{\uparrow 20}^{11} \oplus c_{\uparrow 20})) \oplus \\ S^{-1}(u_{\uparrow 16}^{10} \oplus S^{-1}(u_{\uparrow 5}^{11} \oplus c_{\uparrow 5}) \oplus S^{-1}(u_{\uparrow 16}^{11} \oplus c_{\uparrow 16}) \oplus S^{-1}(u_{\uparrow 22}^{11} \oplus c_{\uparrow 22})) \end{array} \right) \\ S^{-1} \left(\begin{array}{l} u_{\uparrow 23}^9 \oplus \\ S^{-1}(u_{\uparrow 9}^{10} \oplus S^{-1}(u_{\uparrow 9}^{11} \oplus c_{\uparrow 9}) \oplus S^{-1}(u_{\uparrow 15}^{11} \oplus c_{\uparrow 15}) \oplus S^{-1}(u_{\uparrow 16}^{11} \oplus c_{\uparrow 16})) \oplus \\ S^{-1}(u_{\uparrow 23}^{10} \oplus S^{-1}(u_{\uparrow 12}^{11} \oplus c_{\uparrow 12}) \oplus S^{-1}(u_{\uparrow 23}^{11} \oplus c_{\uparrow 23}) \oplus S^{-1}(u_{\uparrow 29}^{11} \oplus c_{\uparrow 29})) \oplus \\ S^{-1}(u_{\uparrow 28}^{10} \oplus S^{-1}(u_{\uparrow 1}^{11} \oplus c_{\uparrow 1}) \oplus S^{-1}(u_{\uparrow 18}^{11} \oplus c_{\uparrow 18}) \oplus S^{-1}(u_{\uparrow 28}^{11} \oplus c_{\uparrow 28})) \end{array} \right) \end{array} \right).$$

This attack is building upon the methodology presented in Section 3.2. The process begins by populating a binary array A of size 2^{88} , which stores the parity of the collected ciphertexts $\mathcal{C}$ across 20 nibble positions:

$$A[i] = \begin{cases} 1, & \text{if } |\{c \in \mathcal{C} : c_{\uparrow 0,6,23,5,11,28,13,24,30,8,14,31,3,20,16,22,9,15,12,29} = i\}| \text{ is odd} \\ 0, & \text{if } |\{c \in \mathcal{C} : c_{\uparrow 0,6,23,5,11,28,13,24,30,8,14,31,3,20,16,22,9,15,12,29} = i\}| \text{ is even} \end{cases}$$

Similarly to the previous attack, we combine the partial sum technique with the FHT to efficiently isolate a series of intermediate variables, in this case $a_0, \dots, a_8$ as defined

741 in Equation (5).

$$\begin{aligned}
a_0 &= S^{-1}(u_{\uparrow 0}^{11} \oplus c_{\uparrow 0}) \oplus S^{-1}(u_{\uparrow 6}^{11} \oplus c_{\uparrow 6}) \oplus S^{-1}(u_{\uparrow 23}^{11} \oplus c_{\uparrow 23}) \\
a_1 &= S^{-1}(u_{\uparrow 5}^{11} \oplus c_{\uparrow 5}) \oplus S^{-1}(u_{\uparrow 11}^{11} \oplus c_{\uparrow 11}) \oplus S^{-1}(u_{\uparrow 28}^{11} \oplus c_{\uparrow 28}) \\
a_2 &= S^{-1}(u_{\uparrow 13}^{11} \oplus c_{\uparrow 13}) \oplus S^{-1}(u_{\uparrow 24}^{11} \oplus c_{\uparrow 24}) \oplus S^{-1}(u_{\uparrow 30}^{11} \oplus c_{\uparrow 30}) \\
a_3 &= S^{-1}(u_{\uparrow 8}^{11} \oplus c_{\uparrow 8}) \oplus S^{-1}(u_{\uparrow 14}^{11} \oplus c_{\uparrow 14}) \oplus S^{-1}(u_{\uparrow 31}^{11} \oplus c_{\uparrow 31}) \\
a_4 &= S^{-1}(u_{\uparrow 3}^{11} \oplus c_{\uparrow 3}) \oplus S^{-1}(u_{\uparrow 13}^{11} \oplus c_{\uparrow 13}) \oplus S^{-1}(u_{\uparrow 20}^{11} \oplus c_{\uparrow 20}) \\
a_5 &= S^{-1}(u_{\uparrow 5}^{11} \oplus c_{\uparrow 5}) \oplus S^{-1}(u_{\uparrow 16}^{11} \oplus c_{\uparrow 16}) \oplus S^{-1}(u_{\uparrow 22}^{11} \oplus c_{\uparrow 22}) \\
a_6 &= S^{-1}(u_{\uparrow 9}^{11} \oplus c_{\uparrow 9}) \oplus S^{-1}(u_{\uparrow 15}^{11} \oplus c_{\uparrow 15}) \oplus S^{-1}(u_{\uparrow 16}^{11} \oplus c_{\uparrow 16}) \\
a_7 &= S^{-1}(u_{\uparrow 12}^{11} \oplus c_{\uparrow 12}) \oplus S^{-1}(u_{\uparrow 23}^{11} \oplus c_{\uparrow 23}) \oplus S^{-1}(u_{\uparrow 29}^{11} \oplus c_{\uparrow 29}) \\
a_8 &= S^{-1}(u_{\uparrow 1}^{11} \oplus c_{\uparrow 1}) \oplus S^{-1}(u_{\uparrow 18}^{11} \oplus c_{\uparrow 18}) \oplus S^{-1}(u_{\uparrow 28}^{11} \oplus c_{\uparrow 28})
\end{aligned} \tag{5}$$

742 The core of this attack is a nine-step iterative procedure designed to systematically
743 reduce the 2^{88} -bit array A . The goal of this phase is to produce a smaller array, $\mathcal{A}$,
744 which stores the parity of occurrences for the tuple $(a_8, a_7, \dots, a_0)$ similarly to the
745 previous attack described in Section 3.2. A detailed breakdown of the complexity
746 of each of these nine steps is provided in Table 9. Note that the last five steps are
747 computing FHT of size 2^8 instead of 2^{12} , as in each of these steps, one of the three
748 required key nibbles has already been determined from a previous stage of the attack.
749 The corresponding indicator function for these steps takes the form:

$$I_\gamma(x_0, x_1) := \begin{cases} 1, & \text{if } S^{-1}(x_0) \oplus S^{-1}(x_1) = \gamma \\ 0, & \text{otherwise} \end{cases}$$

750 Upon completion of these nine reduction steps, we are left with the array $\mathcal{A}$ of size
751 2^{36} and we have to compute the following sum given in Equation (6).

$$\bigoplus_{a \in \mathcal{A}} S^{-1} \left(\begin{aligned} &u_{\uparrow 0}^8 \oplus \\ &S^{-1}(u_{\uparrow 0}^9 \oplus S^{-1}(u_{\uparrow 0}^{10} \oplus a_0) \oplus S^{-1}(u_{\uparrow 5}^{10} \oplus a_1) \oplus S^{-1}(u_{\uparrow 24}^{10} \oplus a_2)) \oplus \\ &S^{-1}(u_{\uparrow 8}^9 \oplus S^{-1}(u_{\uparrow 8}^{10} \oplus a_3) \oplus S^{-1}(u_{\uparrow 13}^{10} \oplus a_4) \oplus S^{-1}(u_{\uparrow 16}^{10} \oplus a_5)) \oplus \\ &S^{-1}(u_{\uparrow 23}^9 \oplus S^{-1}(u_{\uparrow 9}^{10} \oplus a_6) \oplus S^{-1}(u_{\uparrow 23}^{10} \oplus a_7) \oplus S^{-1}(u_{\uparrow 28}^9 \oplus a_8)) \end{aligned} \right) \tag{6}$$

752 This array is structurally identical to the initial array A from the 10-round attack
753 detailed in Section 3.2. Consequently, the problem of recovering the final key by pro-
754 cessing $\mathcal{A}$ to compute the XOR sum, given in Equation (6), is equivalent to the entire
755 attack presented in that section. As established previously, the complexity of this final
756 key-recovery phase is $2^{57.4}$ or $2^{54.476}$ (using the packing technique). The total com-
757 plexity of the attack is therefore the sum of the initial nine-step reduction phase and
758 this final phase. This can be expressed as:

$$(12 \cdot 2^{12}) + (8 \cdot 2^8) + (2^{80} + 2^{84} + 2^{88} + 2^{92}) \cdot 2^{16.64} +$$

Table 9 Complexity analysis of the first 9 steps of the attack on 11 rounds.

Step	Guess	Conv. Domain	Time (Integer Arithmetic)	Memory (bits)
1	-	$C_{\uparrow 0,6,23}$	$2^{80} \times 2^{16.64}$	2^{92}
2	$u_{\uparrow 0,6,23}^{11}$	$C_{\uparrow 5,11,28}$	$2^{84} \times 2^{16.64}$	2^{92}
3	$u_{\uparrow 0,6,23,5,11,28}^{11}$	$C_{\uparrow 13,24,30}$	$2^{88} \times 2^{16.64}$	2^{84}
4	$u_{\uparrow 0,6,23,5,11,28,13,24,30}^{11}$	$C_{\uparrow 8,14,31}$	$2^{92} \times 2^{16.64}$	2^{76}
5	$u_{\uparrow 0,6,23,5,11,28,13,24,30,8,14,31}^{11}$	$C_{\uparrow 3,20}$	$2^{100} \times 2^{12.08}$	2^{68}
6	$u_{\uparrow 0,6,23,5,11,28,13,24,30,8,14,31,3,20}^{11}$	$C_{\uparrow 16,22}$	$2^{104} \times 2^{12.08}$	2^{64}
7	$u_{\uparrow 0,6,23,5,11,28,13,24,30,8,14,31,3,20,16,22}^{11}$	$C_{\uparrow 9,15}$	$2^{108} \times 2^{12.08}$	2^{60}
8	$u_{\uparrow 0,6,23,5,11,28,13,24,30,8,14,31,3,20,16,22,9,15}^{11}$	$C_{\uparrow 12,29}$	$2^{112} \times 2^{12.08}$	2^{56}
9	$u_{\uparrow 0,6,23,5,11,28,13,24,30,8,14,31,3,20,16,22,9,15,12,29}^{11}$	$C_{\uparrow 1,18}$	$2^{116} \times 2^{12.08}$	2^{52}

Table 10 Progressive ZeroSum checks for the 11-round attack using 7-round distinguisher

Step	1	2	3	4
Selected Column $x_{\uparrow ?}^8$	0	8	23	31
$\log_2(\mathcal{L})$	100	96	60	24
Candidate Size (Bits)	140	176	180	184

$$(2^{100} + 2^{104} + 2^{108} + 2^{112} + 2^{116}) \cdot 2^{12.08} + 2^{88} \cdot 2^{57.4} \approx 2^{145.4}.$$

759 If we consider the packing technique with computations over 64-bit words and we can
760 pack 4 FHTs of size 2^{12} , 8 FHTs of size 2^8 , and 16 FHTs of size 2^4 into a single word.
761 Then the total complexity of the attack reduces to:

$$(12 \cdot 2^{12}) + (8 \cdot 2^8) + (4 \cdot 2^4) + (2^{80} + 2^{84} + 2^{88} + 2^{92}) \cdot 2^{-2} \cdot 2^{16.64} + \\ (2^{100} + 2^{104} + 2^{108} + 2^{112} + 2^{116}) \cdot 2^{-3} \cdot 2^{12.08} + 2^{88} \cdot 2^{54.76} \approx 2^{142.76}.$$

762 **Master key recovery.** To recover the master key, we utilize 8 independent cubes
763 of dimension 116 with a total of 40 bits filtering conditions, resulting in a total data
764 complexity of 2^{119} . We focus on recovering the complete round keys for the 10-th and
765 11-th rounds. To recover the full round keys of the 10-th and 11-th rounds, we apply
766 ZeroSum checks across 4 selected columns, as specified in Table 10.

767 After performing these ZeroSum checks, we expect to obtain approximately 2^{24}
768 candidates for 48 bits are from u^{10} and 104 bits are from u^{11} . So we need to recover
769 remaining 80 bits of u^{10} and 24 bits of u^{11} . Thus, we expect to be left with about
770 $2^{24+104} = 2^{128}$ candidates which we then brute-force.

771 The total time complexity includes 4 ZeroSum-based attacks and a final brute-force
772 over 2^{128} candidates: $4 \times 8 \times 2^{145.4} + 2^{128} \approx 2^{150.4}$ or $4 \times 8 \times 2^{142.76} + 2^{128} \approx 2^{147.76}$ if

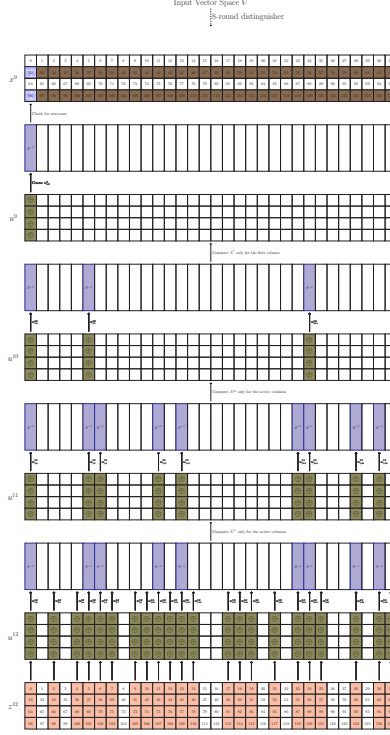


Fig. 5 Overview of the 12-round attack on ARADI using 8-round distinguisher

we consider packing techniques. The memory complexity is dominated by maximum size of $\mathcal{L}$ which is $2^{100} \cdot 140 \approx 2^{107.13}$ bits.

Remark 6. To compute a single column at round 8, data from only 22 columns of the ciphertext is required. While each column in the round function depends on three columns from the following round, many of these dependencies overlap, particularly in the final round. As a result, even after four rounds of backward computation, it is not necessary to process the entire ciphertext state. This behavior directly stems from the slow diffusion property of the cipher’s linear layer. This structural weakness plays a central role in enabling our 11-round and 12-round key-recovery attacks, which are discussed later.

3.5 A 12-Round Attack Using 8-Round Distinguisher

We finally discuss our main result — an attack on 12 rounds of ARADI. It uses an 8-round ZeroSum distinguisher (given in Table 4), and we extend it by four additional rounds to mount a key-recovery attack on 12 rounds of ARADI. We omit the low-level details of the attack, as the strategy closely parallels that of the 11-round attack described in Section 3.4.

The overview of the attack is provided in Figure 5. To recover the full master key, we employ 7 independent cubes, resulting in a total data complexity of $2^{126.8}$.

Table 11 Progressive ZeroSum checks for the 12-round attack using 8-round distinguisher

Step	1	2	3	4	5	6	7	8
Selected Column $x_{\uparrow?}^9$	0	19	30	11	6	25	1	20
$\log_2(\mathcal{L})$	108	116	108	99	78	56	30	0
Candidate Size (Bits)	140	180	204	227	238	248	254	256

Our focus is on recovering the complete round keys for the 10-th and 11-th rounds. To achieve this, we apply ZeroSum checks across 8 strategically selected columns, as detailed in Table 11. We also use the linear dependency between the round keys to filter key-bits as discussed in previous attacks.

One important distinction between the 11-round and 12-round attacks lies in the diffusion behavior observed during backward computation. Our analysis shows that recovering columns $u_{\uparrow 0}^9$ through $u_{\uparrow 15}^8$ requires access to only 22 columns of ciphertext, while columns $u_{\uparrow 16}^9$ through $u_{\uparrow 31}^9$ require 23 columns due to slightly wider dependency. As a result, when we perform ZeroSum checks on columns $u_{\uparrow 19}^9$, $u_{\uparrow 30}^9$, $u_{\uparrow 25}^9$, and $u_{\uparrow 20}^9$, the corresponding FHT computations must consider data array of size 2^{92} . In contrast, the other four steps require 2^{88} , as in the 11-round attack. Thus, the total complexity of these steps are $4 \cdot 8 \cdot 2^{145.4} + 4 \cdot 8 \cdot 2^{149.4} \approx 2^{154.48}$ or $4 \cdot 8 \cdot 2^{142.76} + 4 \cdot 8 \cdot 2^{146.76} \approx 2^{151.8}$ integer additions/subtractions.

After performing these four ZeroSum checks, we expect to obtain an unique candidate for 40 bits of u^{10} and 92 bits of u^{11} . This leaves us with 88 bits of u^{10} and 36 bits of u^{11} yet to be guessed, which we brute-force. Therefore, the total time complexity is upper bounded by the 8 ZeroSum-based attacks, which is $2^{154.96}$ or $2^{142.3}$ (using packing). The memory complexity is dominated by the memory requirements to store $\mathcal{L}$. The memory complexity is dominated by the memory requirements to store $\mathcal{L}$. Storing the largest candidate list $\mathcal{L}$, which contains up to 2^{116} entries, requires at most $2^{116} \cdot 180 \approx 2^{123.5}$ bits.

4 Conclusion

In this paper, we investigated key-recovery attacks on the ARADI cipher. We presented attacks up to 12 rounds based on ZeroSum distinguishers, using the combined partial sum and FHT-based key-recovery technique. We demonstrated how the low diffusion properties of its linear layers can be effectively exploited to recover round key material. Our analysis shows that to partially decrypt one active column through 3.5 rounds, we need at most 22 or 23 active columns. This highlights the importance of carefully analyzing the diffusion behavior of linear layers across several rounds during the design of ciphers. A key strength of our approach lies in the generality of the key-guessing strategy, which is not limited to ZeroSum distinguishers. This methodology can be adapted to other classes of distinguishers, such as differential and linear ones, making it a versatile tool in the broader context of symmetric cryptanalysis. An interesting direction for future work is the development of new distinguishers with sparse *activity*

825 *pattern* in key-recovery, such that it is possible to minimize the number of round key
826 columns that need to be guessed during partial encryption/decryption.

827 **Acknowledgements.** The first author was supported in part by the Israeli Science
828 Foundation through grant 1437/25. The second author was supported by the European
829 Research Council starting researcher grant ReSCALE, number 101041545. Views and
830 opinions expressed are however those of the author(s) only and do not necessarily
831 reflect those of the granting agency.

832 References

- 833 [ABD⁺23] Roberto Avanzi, Subhadeep Banik, Orr Dunkelman, Maria Eichlseder,
834 Shibam Ghosh, Marcel Nageler, and Francesco Regazzoni. The QARMAv2
835 Family of Tweakable Block Ciphers. *IACR Trans. Symmetric Cryptol.*,
836 2023(3):25–73, 2023.
- 837 [ADG24] Roberto Avanzi, Orr Dunkelman, and Shibam Ghosh. A note on ARADI
838 and LLAMA. Cryptology ePrint Archive, Paper 2024/1328, 2024.
- 839 [AM09] Jean-Philippe Aumasson and Willi Meier. Zero-sum distinguishers for
840 reduced Keccak-f and for the core functions of Luffa and Hamsi. *rump ses-*
841 *sion of Cryptographic Hardware and Embedded Systems (CHES)*, 2009:67,
842 2009.
- 843 [AMS⁺22] Roberto Avanzi, Ionut Mihalcea, David Schall, Andreas Sandberg, and
844 Héctor Montaner. Cryptographic protection of random access memory:
845 How inconspicuous can hardening against the most powerful adversaries
846 be? *IACR Cryptology ePrint Archive, Paper 2022/1472*, 2022.
- 847 [Ava17] Roberto Avanzi. The QARMA Block Cipher Family. Almost MDS Mat-
848 rices Over Rings With Zero Divisors, Nearly Symmetric Even-Mansour Con-
849 structions With Non-Involutory Central Rounds, and Search Heuristics for
850 Low-Latency S-Boxes. *IACR Trans. Symmetric Cryptol.*, 2017(1):4–44,
851 2017.
- 852 [BC10] Christina Boura and Anne Canteaut. Zero-Sum Distinguishers for Iterated
853 Permutations and Application to Keccak-f and Hamsi-256. In *Proceedings*
854 *of Selected Areas in Cryptography (SAC)*, volume 6544 of *Lecture Notes*
855 *in Computer Science*, pages 1–17. Springer, 2010.
- 856 [BCC11] Christina Boura, Anne Canteaut, and Christophe De Cannière. Higher-
857 Order Differential Properties of Keccak and Luffa. In *Proceedings of Fast*
858 *Software Encryption (FSE)*, volume 6733 of *Lecture Notes in Computer*
859 *Science*, pages 252–269. Springer, 2011.
- 860 [BCG⁺12] Julia Borghoff, Anne Canteaut, Tim Güneysu, Elif Bilge Kavun, Miroslav
861 Knezevic, Lars R. Knudsen, Gregor Leander, Ventzislav Nikov, Christof
862 Paar, Christian Rechberger, Peter Rombouts, Søren S. Thomsen, and
863 Tolga Yalçin. PRINCE - a low-latency block cipher for pervasive com-
864 puting applications - extended abstract. In *Advances in Cryptology -*
865 *Proceedings of ASIACRYPT*, pages 208–225, 2012.
- 866 [BDG⁺25] Yanis Belkheyar, Patrick Derbez, Shibam Ghosh, Gregor Leander, Sil-
867 via Mella, Léo Perrin, Shahram Rasoolzadeh, Lukas Stennes, Siwei Sun,

- 868 Gilles Van Assche, and Damian Vizár. Chilow and chichi: New construc-
869 tions for code encryption. In *Advances in Cryptology – Proceedings of*
870 *EUROCRYPT*, volume 15601 of *Lecture Notes in Computer Science*, pages
871 212–243. Springer, 2025.
- 872 [BEK⁺20] Dusan Bozilov, Maria Eichlseder, Miroslav Knezevic, Baptiste Lambin,
873 Gregor Leander, Thorben Moos, Ventzislav Nikov, Shahram Rasoolzadeh,
874 Yosuke Todo, and Friedrich Wiemer. PRINCEv2 - more security for
875 (almost) no overhead. In *Proceedings of Selected Areas in Cryptography*
876 *(SAC)*, volume 12804 of *LNCS*, pages 483–511. Springer, 2020.
- 877 [BFG⁺24] Emanuele Bellini, Mattia Formenti, David Gérault, Juan Grados, Anna
878 Hambitzer, Yun Ju Huang, Paul Huynh, Mohamed Rachidi, Raghvendra
879 Rohit, and Sharwan K. Tiwari. Claasping ARADI: automated analysis of
880 the ARADI block cipher. In *INDOCRYPT (2)*, volume 15496 of *Lecture*
881 *Notes in Computer Science*, pages 90–113, 2024.
- 882 [BJK⁺16] Christof Beierle, Jérémy Jean, Stefan Kölbl, Gregor Leander, Amir
883 Moradi, Thomas Peyrin, Yu Sasaki, Pascal Sasdrich, and Siang Meng Sim.
884 The SKINNY family of block ciphers and its low-latency variant MAN-
885 TIS. In *Advances in Cryptology – Proceedings of CRYPTO*, pages 123–153,
886 2016.
- 887 [BRRT25] Emanuele Bellini, Mohamed Rachidi, Raghvendra Rohit, and Sharwan K.
888 Tiwari. On the structural properties of toffoli gate composition in ARADI:
889 implications for algebraic distinguishers. In *ACNS (2)*, volume 15826 of
890 *Lecture Notes in Computer Science*, pages 400–425. Springer, 2025.
- 891 [CGL⁺23] Federico Canale, Tim Güneysu, Gregor Leander, Jan Philipp Thoma,
892 Yosuke Todo, and Rei Ueno. SCARF - A Low-Latency Block Cipher for
893 Secure Cache-Randomization. In *USENIX Security Symposium*, pages
894 1937–1954. USENIX Association, 2023.
- 895 [CSQ07] Baudoin Collard, F. X. Standaert, and Jean-Jacques Quisquater. Improv-
896 ing the Time Complexity of Matsui’s Linear Cryptanalysis. In *Proceedings*
897 *of Information Security and Cryptology (ICISC)*, volume 4817 of *Lecture*
898 *Notes in Computer Science*, pages 77–88, 2007.
- 899 [DE25] Patrick Derbez and Marie Euler. Minimalist model for impossible
900 differentials. Cryptology ePrint Archive, Paper 2025/1552, 2025.
- 901 [DG25] Orr Dunkelman and Shibam Ghosh. Improved key-recovery attacks on
902 ARADI. Cryptology ePrint Archive, Paper 2025/1227, 2025.
- 903 [DGK⁺24] Orr Dunkelman, Shibam Ghosh, Nathan Keller, Gaëtan Leurent, Avichai
904 Marmor, and Victor Mollimard. Partial Sums Meet FFT: Improved Attack
905 on 6-Round AES. In *Advances in Cryptology – Proceedings of EURO-*
906 *CRYPT*, volume 14651 of *Lecture Notes in Computer Science*, pages
907 128–157. Springer, 2024.
- 908 [DR00] Joan Daemen and Vincent Rijmen. Rijndael for AES. In *AES Can-*
909 *didate Conference*, pages 343–348. National Institute of Standards and
910 Technology, 2000.
- 911 [FKL⁺00] Niels Ferguson, John Kelsey, Stefan Lucks, Bruce Schneier, Michael Stay,
912 David A. Wagner, and Doug Whiting. Improved Cryptanalysis of Rijndael.

913 In *Proceedings of Fast Software Encryption (FSE)*, volume 1978 of *Lecture*
914 *Notes in Computer Science*, pages 213–230. Springer, 2000.

915 [GMNP25] Shibam Ghosh, Bastien Michel, and María Naya-Plasencia. Differential-
916 MITM attack on 14-round ARADI. Cryptology ePrint Archive, Paper
917 2025/1918, 2025.

918 [GMW24] Patricia Greene, Mark Motley, and Bryan Weeks. ARADI and LLAMA:
919 Low-latency cryptography for memory encryption. Cryptology ePrint
920 Archive, Paper 2024/1240, 2024. <https://eprint.iacr.org/2024/1240>.

921 [HJ25] Xichao Hu and Lin Jiao. Partial-guess, pre-sieve, greedy-search - new
922 unified key recovery framework of impossible boomerang attacks: Full-
923 round attack on ARADI. Cryptology ePrint Archive, Paper 2025/056,
924 2025.

925 [HLM⁺20] Yonglin Hao, Gregor Leander, Willi Meier, Yosuke Todo, and Qingju
926 Wang. Modeling for Three-Subset Division Property Without Unknown
927 Subset - Improved Cube Attacks Against Trivium and Grain-128AEAD.
928 In *Advances in Cryptology – Proceedings of EUROCRYPT*, volume 12105
929 of *Lecture Notes in Computer Science*, pages 466–495. Springer, 2020.

930 [HSH⁺08] J. Alex Halderman, Seth D. Schoen, Nadia Heninger, William Clarkson,
931 William Paul, Joseph A. Calandrino, Ariel J. Feldman, Jacob Appel-
932 baum, and Edward W. Felten. Lest we remember: Cold boot attacks on
933 encryption keys. In Paul C. van Oorschot, editor, *Proceedings of the 17th*
934 *USENIX Security Symposium, July 28-August 1, 2008, San Jose, CA,*
935 *USA*, pages 45–60. USENIX Association, 2008.

936 [KH24] Donggeun Kwon and Seokhie Hong. Side-channel attack on ARADI.
937 Cryptology ePrint Archive, Paper 2024/1987, 2024.

938 [KKL⁺24] Sunyeop Kim, Insung Kim, Dongjae Lee, Deukjo Hong, Jaechul Sung, and
939 Seokhie Hong. Byte-wise equal property of ARADI. Cryptology ePrint
940 Archive, Paper 2024/1772, 2024.

941 [Lai94] Xuejia Lai. Higher Order Derivatives and Differential Cryptanalysis. In
942 *Communications and Cryptography: Two Sides of One Tapestry*, pages
943 227–233. Springer, 1994.

944 [LDF20] Baptiste Lambin, Patrick Derbez, and Pierre-Alain Fouque. Linearly
945 equivalent s-boxes and the division property. *Des. Codes Cryptogr.*,
946 88(10):2207–2231, 2020.

947 [TA14] Yosuke Todo and Kazumaro Aoki. FFT Key Recovery for Integral Attack.
948 In *Proceedings of Cryptology and Network Security (CANS)*, volume 8813
949 of *Lecture Notes in Computer Science*, pages 64–81. Springer, 2014.